# 线上观影厅项目文档

项目小组 第四小组

小组成员 刘帅、何鑫宇、苏成、黄昆、钟彧、谢延盛

联系方式 19232325727

重庆师范大学软件工程系

# 摘要

项目背景 随着生活范围扩大，异地情侣与亲友因地理阻隔，缺乏能够共同参与的高质量社交娱乐活动 。虽然现有市场存在观影解决方案，但普遍面临版权限制、会员收费、操作繁琐以及运维成本高等痛点，难以满足用户对于“轻量、免费、自主片源”的陪伴式观影需求 。

项目目标 本项目旨在开发一个基于 Web 技术的全栈“线上观影厅”应用，仿照 B 站放映厅功能，打造免费、无广告且开箱即用的解决方案 。平台致力于实现视频播放的实时同步与低延迟互动（弹幕、聊天），并支持用户自主上传或提供视频源以解决版权限制 。最终目的是消除异地隔阂，完美复刻线下“客厅沙发观影”的沉浸式社交体验 。

# 目录


摘要.....2

第 1 章 立项.....7

1.1. 项目起源与提案.....7

1. 项目起源.....7

2. 项目提案.....7

2.1. 要解决的问题.....7

2.2. 目标用户.....7

2.3. 解决方案.....7

2.4. 当前做的原因.....7

1.2. Business Case.....8

1. 项目价值与收益（Value & Benefits）：.....8

1.1. 目标市场.....8

1.2. 解决真实痛点.....8

1.3. 技术栈验证.....8

1.4. 作品集价值.....8

1.5. 竞争优势.....8

2. 成本与资源头投入（Cost & Resources）：.....8

2.1. 技术成本.....8

2.2. 运营成本.....8

1.3. 项目可行性.....8

1. 市场可行性.....8

1.1. 目标用户.....8

1.2. 需求验证.....8

1.3. 市场空间.....8

2. 技术可行性.....9

2.1. 团队技术能力.....9

2.2. 关键技术难点及解决方案.....9

2.3. 技术选型合理性.....9

第 2 章 愿景.....10

2.1. 问题陈述.....10

1. 问题一：现有方案的局限性.....10

2. 问题二：缺乏专门的解决方案.....10

3. 更多.....10

2.2. 涉众与用户.....12

1. 涉众.....12

2. 用户.....13

2.3. 关键涉众和用户的需要.....13

1. 关键涉众的需要.....13

1.1. Developers 的需要.....13

1.2. Users 的需要.....13

2. 关键用户的需要.....13

2.1. 异地情侣的需要.....13

3. 关键用户的需要与特性分析.....13
2.4. 产品概述.....14
1. 产品定位陈述.....14
2. 完整的产品概述.....14
2.5. 产品特性.....15
2.6. 其他产品需求.....15
1. 性能需求.....15
2. 兼容性需求.....15
3. 可维护性需求.....15
第 3 章 用况建模.....16
3.1. 识别参与者.....16
1. 观众（Audience）.....16
2. 放映厅创建者（Host / Presenter）.....16
3. 放映控制者（Screening Controller）.....16
4. 视频库维护者（Video Library Maintainer）.....16
3.2. 识别用例.....17
1. UC-01 参与社交影院（Participate in social cinema）.....17
2. UC-02 维护视频库（Maintain video library）.....17
3.3. 用例图.....17
3.4. 术语表.....18
3.5. 用例描述.....20
1. UC-01 参与社交影院（Participate in social cinema）.....20
1.1. 参与者.....20
1.2. 前置条件.....20
1.3. 基本流.....20
1.4. 备选流.....22
1.5. 子流.....30
2. UC-02 维护视频库（Maintain video library）.....31
2.1. 参与者.....31
2.2. 前置条件.....31
2.3. 基本流.....31
2.4. 备选流.....33
2.5. 后置条件.....35
第 4 章 需求分析.....36
4.1. 健壮性分析.....36
1. 边界类.....36
2. 控制类.....36
3. 实体类.....37
4.2. 顺序模型.....38
1. 创建-设置片源 顺序图.....38
2. 观众加入-观众初始化 顺序图.....42
3. 同步放映-聊天 顺序图.....44
4. 解散房间 顺序图.....48
4.3. 状态模型.....49

1. UML 状态机图.....49
2. 状态模型的核心状态分类.....51
3. 状态流转的关键逻辑.....51
4. 状态模型的核心特点.....52
4.4. 完善分析类图.....52
第 5 章 架构设计.....59
5.1. 架构目标与约束.....59
1. 架构目标.....59
2. 架构约束.....59
5.2. 软件架构风格说明.....60
1. 前后端分离架构（SPA+RESTful/WebSocket）.....60
2. 基于 DDD 思想的分层设计.....60
3. 模块化单体架构与垂直分区.....62
4. 数据管理策略选择——内存存储+Repository 抽象.....63
5.3. 系统逻辑结构.....63
1. 将系统拆分为子系统.....63
1.1. 子系统拆分与核心定义.....63
1.2. 分层设计.....64
1.3. 分区设计.....65
1.4. 组合分层和分区.....65
2. 前端包结构.....66
2.1. 表现层.....67
2.2. 控制层.....68
2.3. 架构特点.....68
3. 后端包结构.....68
3.1. 控制层.....68
3.2. 实体层.....69
3.3. 架构特点.....69
5.4. 系统组件结构.....69
1. 核心组件说明.....70
2. 组件间通信方式.....70
3. 第三方依赖识别.....70
5.5. 物理部署结构.....70
1. 部署拓扑概述.....71
2. 节点运行环境说明.....71
3. 网络通信协议与端口规划.....72
4. 部署与运维约束.....73
5.6. 并发与分布处理.....73
1. 识别内部并发性.....73
2. 定义并发任务.....73
2.1. 并发任务划分.....73
2.2. 控制线程处理并发.....74
5.7. 处理器分配.....75
1. 系统部署架构概述.....75

2. 子系统的处理需求.....75
3. 分配策略.....76
4. 通信基础设施.....76
第 6 章 详细设计.....78
6.1. 引言.....78
6.2. 类设计与精化.....78
1. 房间管理子系统.....78
2. 同步播放子系统.....80
3. 聊天互动子系统.....83
4. 引入解决方案.....84
5. 应用设计模式.....84
6.3. 动态模型设计.....86
1. 关键用例的序列图.....86
6.4. 接口与人机交互设计.....87
1. 内部接口定义.....87
1.1. HTTP API.....88
1.2. Socket Events：同步控制模块.....96
1.3. Socket Events：聊天互动模块.....97
6.5. 数据管理策略.....98
1. 当前实现.....98
2. 数据模型.....98
3. 设计考虑.....98
4. 后续迭代计划.....98
6.6. 软件实现标准.....98
1. 代码命名规范.....98
1.1. JavaScript/TypeScript 类与文件.....98
1.2. 成员变量与局部变量.....98
1.3. 成员函数（操作）.....99
1.4. Vue.js 组件与属性.....99
1.5. 数据库对象（后续迭代）.....99
2. 可见性与封装标准.....99
2.1. JavaScript/TypeScript 访问控制.....99
2.2. Props：用于将父组件数据传递给子组件.....99
2.3. 项目特点.....101
2.4. 与通用规范的差异.....101
2.5. 模块导出规范.....101
2.6. 项目导出规范总结.....104
后记.....105
参考文献.....106

# 第1章 立项

## 1.1. 项目起源与提案

### 1. 项目起源

External Needs：随着人们生活范围的扩大，朋友、情侣和家人常常因地理位置相隔遥远，无法像过去一样聚在一起看电影、追剧，缺乏共同的社交娱乐活动来维持情感联系。在线上和亲朋好友一起看电影，一定程度上也消除了异地带来的隔阂感。然而现有解决方案存在诸多问题：版权限制、使用门槛高、收费限制、操作复杂等。

### 2. 项目提案

本项目提议开发一个仿照 B 站"放映厅"功能的线上观影平台，打造一个开箱即用、轻量级、无广告的解决方案，让分隔两地的朋友、情侣和家人能够轻松实现"一起看电影"的愿望。

#### 2.1. 要解决的问题

- 异地生活的亲友缺乏高质量、便捷的共享娱乐体验
- 现有的解决方案存在版权限制、操作复杂等
- 多端兼容

#### 2.2. 目标用户

异地情侣、分散在各地的亲密朋友、家庭成员

#### 2.3. 解决方案

- 实时播放同步
- 弹幕和聊天互动
- 片源自行提供（解决片源限制）
- web 端开发（提升便利性、降低跨平台使用门槛）
- 一端搭建，多端享用（降低他人使用门槛）

#### 2.4. 当前做的原因

市场空白：

- 现有的产品因运维成本、片源版权等问题无法实现“免费”、“便利”、“片源自管理”等关键点，有技术但不想付费的用户需要这样一个平台。

技术成熟：

- Socket.IO 实时通信技术已非常成熟
- 用户普遍拥有 ipv6 公网地址。
- Video.js 提供了丰富的可定制的网页播放器。

## 1.2. Business Case

### 1. 项目价值与收益（Value & Benefits）：

#### 1.1. 目标市场

异地情侣、远距离朋友、分隔两地的家人等小圈子人群

#### 1.2. 解决真实痛点

为项目发起者及其朋友提供一个高质量的社交工具，直接提升异地社交的幸福感。

#### 1.3. 技术栈验证

作为一个全栈项目，它将完整覆盖前端（Vue）、后端（Express）、实时通信（Socket.IO）和流媒体服务（Nginx）等关键技术，是绝佳的学习和实践案例。

#### 1.4. 作品集价值

可作为一个功能完整、架构清晰的代表性作品，在未来求职或技术交流中展示全栈开发能力。

#### 1.5. 竞争优势

完全免费、片源自由、零门槛使用、专注观影体验。

### 2. 成本与资源头投入（Cost & Resources）：

#### 2.1. 技术成本

服务器运营、域名注册、SSL 证书、视频暴露等

#### 2.2. 运营成本

需租用一台高带宽云服务器（约 1 核/1GB/100Mbps 配置），用于部署应用和提供视频分发。

## 1.3. 项目可行性

### 1. 市场可行性

#### 1.1. 目标用户

18-35 岁有异地亲友的年轻群体，涵盖学生、职场人士等，该群体线上社交需求旺盛，对新鲜观影模式接受度高

#### 1.2. 需求验证

结合市场反馈，60%以上的异地用户表示有同步观影需求，且 80%用户吐槽现有平台的片源限制与收费问题，需求痛点明确

#### 1.3. 市场空间

当前市场缺乏“轻量、免费、支持自主片源”的陪伴式观影产品，项目具备差异化竞争优势。

## 2. 技术可行性

### 2.1. 团队技术能力

项目团队由6名大学生组成，暂无实时通信/视频处理项目经验，但具备以下优势：

- 熟练掌握JavaScript/Node.js基础（前端已用Vue.js，后端可快速迁移）。
- 通过开源社区资源（如Socket.IO官方文档、FFmpeg教程）可快速上手。
- 团队已掌握基础Web开发（HTML/CSS/JS），学习曲线可控。

### 2.2. 关键技术难点及解决方案

- 同步延迟问题:用Socket.IO广播播放状况事件，服务器统一同步进度。
- 视频格式兼容：用户上传视频 → FFmpeg转码为HLS格式（自动生成.m3u8文件） → Nginx分发流媒体。

### 2.3. 技术选型合理性

- Socket.IO封装了WebSocket/P2P复杂逻辑，提供简单广播API 。
- FFmpeg有现成命令行工具，团队可直接调用，无需从零开发。
- Nginx配置简单（支持HLS流媒体），video.js兼容主流格式，规避了自研播放器的高复杂度。
- Node.js + Express与前端Vue.js同属JS生态，技术兼容好，开发效率高；Express文档丰富，适合新手快速搭建API。

# 第2章愿景

## 2.1. 问题陈述

### 1. 问题一：现有方案的局限性

- **版权问题：**直播间放视频涉及版权风险。
- **配置繁琐：**Video Together 等插件需要双方安装，手机端需特定浏览器。
- **商业限制：**B 站放映厅收费且片源有限。
- **体验问题：**腾讯会议等工具有时长限制和流畅度问题。

| 特性 | 本项目 | B 站放映厅 | Video Together | 视频会议工具 |
|---|---|---|---|---|
| 免费 | √ | X | √ | 部分免费 |
| 片源自由 | √ | X | √ | √ |
| 无需安装 | √ | √（已集成在 B 站） | X | X |
| 手机友好 | √ | √ | X | √ |
| 无时长限制 | √ | X | √ | X |
| 专注观影体验 | √ | √ | √ | X |

### 2. 问题二：缺乏专门的解决方案

市场缺少一个轻量、无广告、操作简单、专门针对小圈子观影场景的解决方案。

### 3. 更多

功能收费问题陈述

| 要素 | 描述 |
|---|---|
| 问题 | 需要收费或充值会员 |
| 影响 | 服务提供方、用户 |
| 结果 | 增加用户观影成本。 |
| 优点 | 我们的项目优点如下：<br>1.开源<br>2.免费无广告<br>3.节省用户开支 |

个人信息问题陈述

| 要素 | 描述 |
| --- | --- |
| 问题 | 大部分平台必须注册账户 |
| 影响 | 服务提供方、用户 |
| 结果 | 用户需要提供个人信息，存在隐私泄露风险；需要注册多余的账号；游客无法使用。 |
| 优点 | 我们的项目优点如下：<br>1.无需注册，简单便捷<br>2.不收集用户个人信息，保护用户个人信息与隐私安全 |

部署配置问题陈述

| 要素 | 描述 |
| --- | --- |
| 问题 | 浏览器插件等安装配置繁琐 |
| 影响 | 用户 |
| 结果 | 用户不得不花费大量时间精力配置观影环境，电脑小白被劝退 |
| 优点 | 我们的项目优点如下：<br>1.部署简单<br>2.用户可以快速上手 |

| 要素 | 描述 |
| --- | --- |
| 问题 | 需要每个用户都安装配置插件 |
| 影响 | 服务提供方、用户 |
| 结果 | 用户花费大量时间，容易被劝退 |
| 优点 | 我们的项目优点如下：<br>1.只需一人部署，其他人直接使用<br>2.节约大量的用户准备时间 |

片源受限问题陈述

| 要素 | 描述 |
| --- | --- |
| 问题 | 因版权受限的片源 |
| 影响 | 服务提供方、用户、影视资源版权方 |
| 结果 | 可能没有用户想要的资源 |
| 优点 | 我们的项目优点如下：<br>1.用户自主上传资源<br>2.私人观影不受版权限制 |

聊天互动问题陈述

| 要素 | 描述 |
| --- | --- |
| 问题 | 基于浏览器插件的观影方式无法发送弹幕/消息 |
| 影响 | 服务提供方、用户 |
| 结果 | 用户观影时不方便互动 |
| 优点 | 我们的项目优点如下：<br>1.用户可以发送弹幕/消息<br>2.增加了观影过程的趣味性<br>3.改善了用户观影体验 |

## 2.2. 涉众与用户

### 1. 涉众

| 涉众 | 涉众类型 | 简要描述 |
| --- | --- | --- |
| 开发者 | Developers | 负责系统实现以及维护的技术人员 |
| 观影者 | Users | 使用系统进行观影的人 |

| | | |
|---|---|---|
| 技术专家 | Authoities | 对系统提供技术支持的人员 |

2. 用户

| | | |
|---|---|---|
| 异地情侣 | 高频用户 | 处于热恋期的异地情侣 |
| 异地家人、朋友 | 中频用户 | 定期进行线上活动的家人朋友 |
| 搭子 | 低频用户 | 偶尔商定一起观看电影等的搭档 |

## 2.3. 关键涉众和用户的需要

### 1. 关键涉众的需要

#### 1.1. Developers 的需要

- 提供明确的功能描述：功能描述必须明确，易于理解
- 提供合理的开发时间安排：时间安排应该合理
- 提供成熟稳定的技术支持：技术必须先进、稳定，确保短时间不会被淘汰
- 提供标准的开发流程：确保开发过程中不会出现冲突和较长的等待
- 提供积极的交流氛围：开发成员之间的交流应该更加积极，以增加开发效率
- 提供结构清晰、模块化的代码设计：减少后期维护成本

#### 1.2. Users 的需要

- 提供流畅的播放体验：播放不应该卡顿、模糊、音画不同轨
- 提供实时的交流：参与者之间的交流应该低延迟、完整
- 提供自由丰富的片源：涵盖常见片源，支持自带片源
- 提供简单的操作：创建观影室、加入观影室的操作无门槛

### 2. 关键用户的需要

#### 2.1. 异地情侣的需要

- 提供快捷方便的操作：随时随地都可以共同观影，操作无门槛
- 提供良好的交流互动体验：通过共同观影，营造陪伴感、消除孤独感、增强亲密感
- 提供自由的片源：片源多样且可以自带片源

### 3. 关键用户的需要与特性分析

异地情侣的需要： 提供快捷方便的操作、提供良好的交流互动体验、提供自由的片源

| 核心需要（Needs） | 对应系统特性（Features） |
|---|---|
| 提供快捷方便的操 | 系统采用轻量化设计 |

| | |
|---|---|
| 作：随时随地都可以共同观影，操作无门槛 | 系统提供极简的用户体验 |
| 提供良好的交流互动体验：通过共同观影，营造陪伴感、消除孤独感、增强亲密感 | 系统提供沉浸式观影 |
| | 系统提供实时的交流互动体验 |
| 提供自由的片源：片源多样且可以自带片源 | 系统提供自由灵活的片源选择 |

## 2.4. 产品概述

### 1. 产品定位陈述

| | |
|---|---|
| for | 需要共同观影的用户 |
| who | 需要同步观影及同步互动 |
| the | 线上观影厅是一个同步、可互动的观影平台 |
| That | 提供同步观影和实时互动功能，让用户能够轻松创建共享的线上观影体验。 |
| Unlike | 操作复杂、功能付费、片源限制 |
| Our product | 免费、片源自由、操作无门槛，专注于提供沉浸式观影体验 |

### 2. 完整的产品概述

“线上观影厅”是一个基于 Web 技术（Vue, Node.js, Socket.IO）开发的全栈应用。它允许用户创建一个私密的观影房间，房间内的管理员可以控制视频的播放、暂停和进度，所有操作都会实时同步给房间内的其他人。应用集成了聊天室功能和弹幕系统，支持用户在观影的同时进行文字交流，旨在完美复刻线下“客厅沙发观影”的社交体验。本项目本质上是一个"聊天室 + 播放同步的流媒体服务"，核心功能包括：

- 实时播放同步
- 弹幕和聊天互动
- 多种视频源支持（上传、直链 URL 等）
- 外挂字幕功能
- 用户管理和房间管理

## 2.5. 产品特性

| Identifier | Description | Priority |
|---|---|---|
| FEAT1 | 系统采用轻量化设计 | Must |
| FEAT2 | 系统提供极简的用户体验 | Must |
| FEAT3 | 系统提供沉浸式观影 | Must |
| FEAT4 | 系统提供自由灵活的片源选择 | Must |
| FEAT5 | 系统提供实时的交流互动体验 | Must |

## 2.6. 其他产品需求

### 1. 性能需求

- 支持 5-10 人同时在线
- 播放同步延迟小于 1 秒
- 消息延迟小于 500 毫秒

### 2. 兼容性需求

- 支持主流现代浏览器
- 良好的 PC 端体验
- 后续支持移动端 APP

### 3. 可维护性需求

- 模块化设计
- 完善的文档
- 清晰的开发规范

# 第3章 用况建模

## 3.1. 识别参与者

根据《用例建模》教材，参与者定义了用户在与系统交互过程中所扮演的角色，且必须位于系统边界之外。

### 1. 观众（Audience）

**描述：**放映活动的参与者。他通过加入放映厅创建者创建的房间，享受与他人完全同步的视听体验，并通过**互动功能**进行交流讨论。

**注：**在特定情况下，观众可被放映厅创建者临时授权，协助执行部分放映管理职能（即成为放映控制者）。

**职责：**

**参与接入：**发现并进入指定的放映房间（房间搜索/房间码/邀请链接等表现形式属于“发现”的实现细节）。

**状态维护：**保持自身客户端与房间“唯一真实状态”对齐（接收同步指令并应用到本地播放器）。

**社交贡献：**发送聊天消息/弹幕/反馈等互动内容，构成社交体验的一部分。

### 2. 放映厅创建者（Host / Presenter）

**描述：**放映活动的组织者与管理者。拥有对房间环境的控制权，能够指定观众为放映控制者。

**职责：**

**活动初始化：**创建观影空间并选定/加载视频资源。

**房间维护：**配置房间属性（公告、密码、人数上限等）及治理不良行为（踢人/禁言/模式切换）。

**指定放映控制者：**根据需要，将观众临时授权为放映控制者。

**解散房间：**在放映活动结束后，放映厅创建者可选择解散房间，释放资源。

### 3. 放映控制者（Screening Controller）

**描述：**放映活动的控制者。负责管理放映状态。

**职责：**

**进度控制：**发出播放控制指令（播放、暂停、跳转进度、倍速等）。

**资源控制：**根据需要，调整房间环境（例如：改变字幕轨道、切换视频源等）。

### 4. 视频库维护者（Video Library Maintainer）

**描述：**负责系统内部片源的引入、维护与治理的运营/技术人员。

**职责：**

上传并创建视频资产，确保片源数量与可用性。

维护视频元数据（名称、简介、分类、标签、封面等），提升检索与选片效率。

根据内容策略/空间策略清理片源：删除过期、重复或不合规的视频。

通过名称检索快速定位目标视频并进行后续维护操作。

## 3.2. 识别用例

### 1. UC-01 参与社交影院（Participate in social cinema）

**涉及参与者：**观众、放映厅创建者、放映控制者。

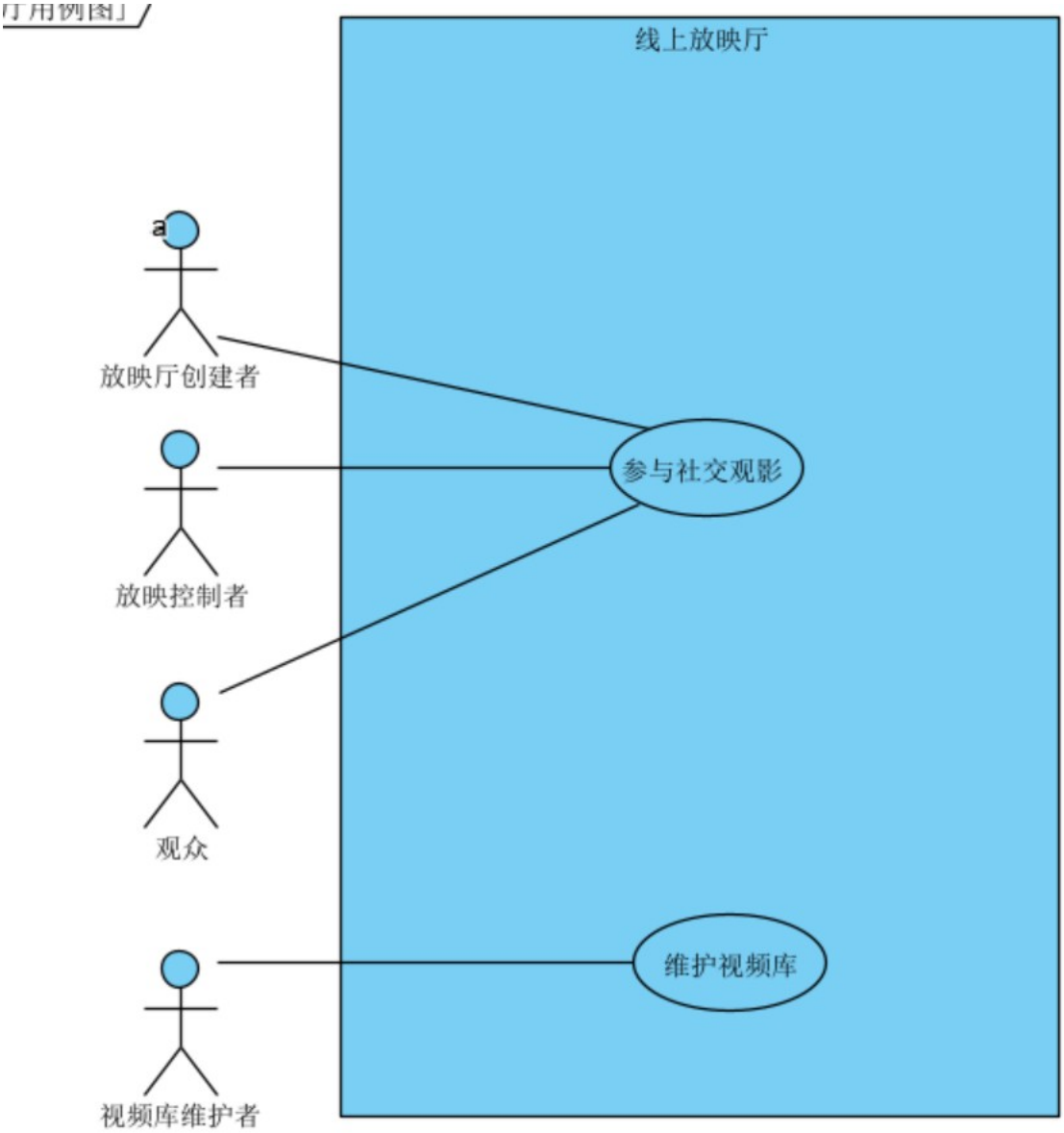
**简略描述：**观众们在一个共同的虚拟房间内（线上观影室）进行共同观影：用户创建房间并加载视频资源，观众加入房间后由系统完成播放状态初始化与持续同步；在放映过程中，放映控制者可以控制放映节奏与内容，观众进行实时讨论互动，放映厅创建者维护房间秩序并在活动结束后解散房间。

### 2. UC-02 维护视频库（Maintain video library）

**涉及参与者：**视频库维护者。

**简略描述：**视频库管理员为保障线上放映厅有充足、优质的片源，通过系统对视频资产进行全生命周期维护的过程。核心操作包括：上传新增视频、编辑视频信息、多条件查找与筛选视频、以及对视频进行删除或下架管理。

## 3.3. 用例图




### 3.4. 术语表

| 术语（Term） | 定义（Definition） |
|---|---|
| **线上观影室** | 一个虚拟房间，承载一次放映活动：成员列表、权限、聊天、以及与放映相关的状态数据。 |
| **线上放映厅** | 同“线上观影室”。 |
| **房间** | 同“线上观影室”。 |
| **房间初始信息** | 房间名称、是否公开、人数上限、是否启用密码、公告 |

| | |
|---|---|
| **房间标识** | 房间号/链接等 |
| **身份校验** | 密码校验、房间是否满员、是否被踢出/拉黑 |
| **初始化数据** | 视频源、字幕、基准时间戳、是否播放、倍速等 |
| **视频库维护者** | 负责系统内部片源的引入、维护与治理的运营/技术人员。 |
| **Sync_Event (同步事件)** | 系统内部传递的一种控制指令包，用于强制统一房间内的播放状态。包含指令类型（SEEK, PAUSE, PLAY, CHANGE_RATE, CHANGE_SOURCE 等）和参数 Payload（时间戳/速率/视频源） |
| **CHANGE_RATE (变速)** | Sync_Event 类型。管理员修改了播放速率。Payload 包含新的速率因子（如 1.5, 2.0）。客户端接收后必须调整本地播放器的 播放速率。 |
| **SEEK (跳转)** | Sync_Event 类型。Payload 包含目标时间戳（毫秒）。 |
| **PAUSE / PLAY (启停)** | Sync_Event 类型。指示播放器暂停或开始。 |
| **CHANGE_SOURCE (换源)** | Sync_Event 类型。管理员更换了播放内容。Payload 包含新的视频 URL、视频标题及初始字幕信息。客户端接收后需重置播放器并加载新资源。 |
| **同步阈值** | 系统允许的最大时间误差值（例如 2000 毫秒）。超过此值的偏差被视为“不同步”，必须触发强制跳转逻辑。 |
| **本地属性** | 指音量和 清晰度。这些属性由观众端独立控制，系统严禁将其同步给其他用户。 |
| **房间状态对象** | 服务器维护的关于房间当前情况的唯一真实数据集合，用于新用户进房时的初始化同步。（包含：正在播放的视频资源标识、全局基准时间戳、当前播放状态、当前字幕轨道） |
| **片源转码服务** | 负责执行耗时的转码任务。本系统依赖该服务完成转码工作，因此被视为**外部参与者**。 |
| **流媒体播放格式** | 一种特殊的视频传输协议，用于通过网络高效、分段地交付内容。本项目特指 **HLS (HTTP Live Streaming)** |
| **视频库** | 系统中所有**视频资产记录**的集合。管理员可以从中检索并选择视频进行播放。 |
| **视频资产列表** | **视频库**的一种视图表现形式。它包含多个**视频资产记录**的摘要，供管理员进行维护操作。 |

| | |
|---|---|
| **视频资产记录** | 系统**持续管理的**关于视频的元数据集合。它包含文件存储位置、处理状态、标题等信息，代表了一个可供流媒体服务使用的资产。 |
| **CHANGE_SUBTITLE (更改字幕)** | Sync_Event 类型。管理员更换了播放内容。Payload 包含新字幕信息。客户端接收后需更改当前播放视频的字幕轨道。 |
| **审计日志** | 系统为每一次控制操作记录的不可篡改的日志，通常包含操作者、操作类型、操作时间、操作参数等信息，用于安全审计和问题追踪 。 |
| **本地属性** | 指音量和清晰度。这些属性由观众端独立控制，系统严禁将其同步给其他用户。 |
| **操作时间戳** | 记录控制操作发生时刻的唯一时间标识。用于解决操作冲突、保证操作顺序以及进行日志记录 。 |
| **状态恢复机制** | 当用户因网络问题暂时断开连接并重连后，系统能够自动将其播放器状态（进度、播放/暂停等）恢复到与当前房间状态对象一致的流程。 |
| **服务选项** | 指系统在观影厅提供的可操作选项，如：发送消息、弹幕开关、离开房间等 |

### 3.5. 用例描述

#### 1. UC-01 参与社交影院（Participate in social cinema）

##### 1.1. 参与者

发起参与者：观众、放映厅创建者、放映控制者。

##### 1.2. 前置条件

- 放映厅创建者已登录系统。
- 系统可用（能够创建房间并提供实时同步与互动服务）。

##### 1.3. 基本流

**{启动活动}**

1.用例启动：放映厅创建者选择“发起社交影院放映”。

2.系统提示放映厅创建者填写房间初始信息（例如：房间名称、人数上限、是否启用密码、公告）。

**{创建线上观影室（初始化房间）}**

3.放映厅创建者提交创建请求及初始配置。

4.系统校验参数合法性（名称格式、人数上限范围等）。

5.系统创建新的线上观影室，生成唯一房间标识（房间号/链接等）。

6.系统初始化视频状态对象（VideoState）：

・当前视频源：空（未加载）

・播放状态：暂停

・进度：0

・倍速：1.0

・字幕：默认/空

・说明：房间的“公告/密码/人数上限/聊天模式”等属于房间配置，由系统保存并在成员加入时可见/可验证，但不属于 VideoState 的播放状态字段。

7.系统要求创建者设置昵称，将该昵称设置为该房间的“放映厅创建者”（最高权限）。

8.系统向放映厅创建者展示房间界面以及房间管理面板。

**{设置片源（检索/选择/加载）}**

9.放映厅创建者在房间内选择“设置片源”，并输入外部视频链接。

10.系统验证该链接格式及可达性。

11.系统更换当前视频源，通知创建者视频已就绪。

a.系统生成 Sync_Event(CHANGE_SOURCE) 并广播给房间内所有在线成员。

b.观众接收事件，自动加载新视频源并重置播放器状态。（新增）

**{观众加入并建立连接}**

12.观众通过房间列表/房间号 发现目标房间，并发起加入请求。

13.系统对观众执行身份校验（如：密码校验、房间是否满员、是否被踢出/拉黑）,通过后要求输入昵称。

14.系统为观众建立实时连接，并将其加入房间在线成员列表。

15.系统向房间内成员广播“观众加入”的系统通知。

**{观众初始化同步（状态对齐）}**

16.系统读取 VideoState 并向新加入观众下发初始化数据（视频源、字幕、基准时间戳、是否播放、倍速等）。

17.观众客户端加载视频资源与字幕，将进度设置到 VideoState 指定的时间点，并设置播放倍速。

18.观众客户端应用本地属性（音量/清晰度/弹幕开关等），且这些变化不影响 VideoState。

19.观众客户端进入同步监听状态，等待并处理后续 Sync_Event。

**{开始放映}**

20.放映厅创建者点击“开始播放”。

21.系统验证放映厅创建者权限与连接状态。

22.系统更新 VideoState 为“播放中”，并写入新的基准时间戳。

23.系统生成 Sync_Event(PLAY)并广播给房间内所有观众。

24.所有观众客户端接收事件并切换为播放状态。

**{放映中控制与全员强制同步（循环）}**

25.在播放过程中，放映厅创建者可重复执行控制操作：播放/暂停、跳转进度、调整倍速（以及字幕切换等）。

26.对于每一次控制操作，系统执行以下可观察行为：

1. 系统验证发起者具有控制权限，且操作参数有效。

2. 系统更新 VideoState（进度/状态/倍速/字幕等），作为新的“唯一真实数据源”。

3. 系统向房间内所有观众发布对应类型的 Sync_Event（PAUSE/PLAY/SEEK/CHANGE_RATE/CHANGE_SUBTITLE）。

4. 观众端依据事件进行强制对齐：若误差超过同步阈值，必须执行强制校准（例如跳转到目标进度）。

**{观众讨论互动（并行持续）}**

27.在放映期间，任一观众可发送聊天消息或弹幕。

28.系统验证消息内容合规（长度/格式/频率/权限）。

29.系统将消息广播给房间内在线成员，并在聊天区展示。

**{结束活动并解散房间}**

30.放映厅创建者选择“解散房间”。

31.系统提示不可恢复的后果并请求确认。

32.放映厅创建者确认。

33.系统强制断开所有成员连接，停止同步广播。

34.系统销毁房间实例及其 VideoState（并按策略清理会话数据）。

35.用例结束。

### 1.4. 备选流

**A1 房间加锁**

在**{启动活动}**、**{开放加入并建立连接}**处，放映厅创建者选择启用密码，则

**{设置房间密码}**

1. 系统提示放映厅创建者设置房间密码，并提示密码规则（例如：4~6 位数字/或至少 6 位可见字符）。

2. 放映厅创建者输入密码并确认。

3.系统验证密码符合规则。

**{保存并反馈}**

4. 系统保存房间密码，并将房间标记为“加锁”。

5.系统提示放映厅创建者“房间已加锁”。

6.用例恢复到基本流的**{创建线上观影室（初始化房间）}**处，继续执行步骤。

**A1.1 密码不符合规则**

在**{设置房间密码}**处，若设置的密码不符合规则，则

**{提示错误}**

1.系统提示“密码不符合规则”。

**{重试}**

2.系统允许放映厅创建者重新输入密码。

3.用例恢复到 A1 的**{设置房间密码}**处，继续执行步骤 2

**A2 观众输入密码加入**

在**{开放加入并建立连接}**处，若观众请求加入，则

**{输入密码}**

1.系统提示观众输入房间密码。

2.观众输入密码。

**{验证并继续}**

3.系统验证密码：

a. 若密码正确，用例恢复到基本流的**{开放加入并建立连接}**处，并继续执行步骤 14（建立实时 连接）。

b. 若密码错误，则跳转到 A2.1。

**A2.1 密码错误**

在**{输入密码}**处，若观众输入的密码错误，则

**{拒绝并提示}**

1.系统拒绝本次加入请求并提示“密码错误”。

**{重试}**

2.系统允许观众重新输入密码，并限制输入次数（例如 3 次）

a.若输入次数超过限制，则系统暂时冻结该观众的加入请求（例如 30 秒），并提示“尝试次数 过多，请稍后再试”。

b.若输入次数未超过限制，则用例恢复到 A2**{验证并继续}**处。

**A3 个性化屏蔽**

在**{观众初始化同步（状态对齐）}**、**{观众讨论互动（并行持续）}**处，观众选择调节本地偏好，则

**{设置本地偏好}**

1.观众调节本地偏好。

2.设置仅对该观众生效：不改变 VideoState，不影响其他观众。

3.用例恢复到基本流的**{观众初始化同步（状态对齐）}**或**{观众讨论互动（并行持续）}**处。

**A4 从内部视频库选择片源**

在基本流**{设置片源}**处，若放映厅创建者或放映控制者不想使用外部链接，则：

1.创建者选择“从视频库检索”。

2.系统展示已上传的视频资产列表。

3.创建者选择目标视频。

4.系统验证视频资产状态（Ready）。

5.用例恢复至基本流步骤 11，系统更新 当前视频源。

**A4.1 处理不可用新片源**

在**{选择新片源}**处，若新片源不可用，则

1.系统提示放映厅创建者“片源不可用”。

2.系统保持当前 VideoState 不变。

3.用例恢复到基本流的**{设置片源（检索/选择/加载）}**处，继续执行步骤 9.

**A5 权限委任**

在任意时刻反映厅创建者打开成员管理，则

**{选择成员与操作}**

1.放映厅创建者选择一名观众并选择操作：“提拔为管理员”或“撤销管理员”。

**{验证操作合法性}**

2.系统验证该操作是否合法（不能撤销放映厅创建者自身；不能对已离开房间的成员操作）。

**{执行并通知}**

3.系统更新该成员权限。

4.系统通知该成员权限变更结果，并向房间发布系统提示（例如“某某已成为管理员”）。

5.用例恢复到基本流，用例继续。

**A5.1 权限变更非法**

在**{验证操作合法性}**处，若操作不合法，则

1.系统提示放映厅创建者“权限变更失败：权限不足或目标无效”。

2.用例恢复到基本流，用例继续。

**A6 秩序维护（踢人/禁言/聊天模式切换）**

在观影厅内出现不良行为，或者放映厅创建者需要临时控制聊天环境时，则

**{选择目标}**

1.放映厅创建者选择目标观众。

**{验证身份}**

2.系统验证目标不是放映厅创建者，且执行者具有对应权限

**{执行操作}**

3.放映厅创建者选择执行以下操作：

a.踢出观众

b.禁言/解除禁言

c.切换聊天模式

4.用例恢复到基本流，用例继续。

**A6.1 踢出观众**

在 A6 秩序维护**{执行操作}**处，若放映厅创建者选择“踢出观众”，则

1.系统将目标观众移出房间并使其立刻离线。

2.系统将目标观众加入房间临时黑名单（例如 5 分钟内不可重新加入）。

3.系统向房间内成员广播系统消息（例如“某用户已被移出房间”）。

**A6.2 禁言/解除禁言**

在 A6 秩序维护**{执行操作}**处，若放映厅创建者选择“禁言/解除禁言”，则

1.放映厅创建者选择目标观众并选择“禁言”或“解除禁言”。

2.若为禁言：放映厅创建者设置禁言时长（例如 10 分钟/永久）。

3.系统验证权限并更新该观众发言状态。

4.系统通知当事观众“已被禁言/已解除禁言”。

**A6.3 切换聊天模式**

在 A6 秩序维护**{执行操作}**处，若放映厅创建者选择“切换聊天模式”，则

1.放映厅创建者选择聊天模式：全员禁言 / 仅管理员可发言 / 自由发言。

2.系统更新房间聊天配置。

3.系统向全员广播“聊天模式已变更”。

**A6.4 治理操作失败**

在 A6 秩序维护**{执行操作}**处，若对应操作执行失败，则

1.系统提示放映厅创建者失败原因（例如“权限不足/目标已离开房间”）。

2.用例恢复到基本流，用例继续。

**A7 房间加入异常**

在基本流**{开放加入并建立连接}**处，若观众加入房间异常，如输入的房间号无效、房间已解散、人数已满、或观众处于黑名单/冷却期等，则

1.系统拒绝加入并提示具体原因。

2.系统不向该观众透露房间内部信息（例如当前播放内容）。

3.该观众用例结束

**A8 视频不可播放**

在基本流**{设置片源（检索/选择/加载）}**、**{观众初始化同步（状态对齐）}**处，若视频不可播放，则

**{判断不可播放阶段}**

1.系统判断不可播放阶段并执行对应操作：

a.放映厅创建者选择片源阶段不可播放

b.观众初始化阶段不可播放

**A8.1 放映厅创建者选择片源阶段不可播放**

在 A8 视频不可播放的**{判断不可播放阶段}**处，若为放映厅创建者选择片源阶段不可播放，则

1.系统提示放映厅创建者“片源不可播放”。

2.用例恢复到基本流的**{设置片源（检索/选择/加载）}**处，继续执行步骤 9。

**A8.2 观众初始化阶段不可播放**

在 A8 视频不可播放的**{判断不可播放阶段}**处，若为观众初始化阶段不可播放，则

1.系统提示该观众“视频加载失败”。

2.系统允许该观众留在房间内，但其播放器处于不可用状态。

3.当放映厅创建者执行换片（见 A4）后，该观众按最新 VideoState 重新初始化。

4.用例恢复到基本流的**{观众初始化同步（状态对齐）}**处，继续执行步骤 20。

**A9 同步失败/网络异常/断线重连**

在基本流**{观众初始化同步（状态对齐）}**、**{放映中控制与全员强制同步（循环）}**处，若观众网络异常导致与系统连接中断，或者长时间未能接收同步事件，则

1.系统将该观众标记为“离线/暂不可达”，并记录异常。

2.观众端尝试重新连接。

3.若重连成功：系统向该观众下发最新 VideoState 并要求其重新初始化。

4.用例恢复到基本流的**{观众初始化同步（状态对齐）}**处，继续执行步骤 20。

5.若重连失败：系统提示“连接失败”，该观众分支结束。

**A10 放映厅创建者断线（管理员接管）**

在基本流**{放映中控制与全员强制同步（循环）}**处，若放映厅创建者连接断开，且房间内存在至少一名管理员，则

**{检测断线}**

1.系统检测到放映厅创建者断线，并将放映厅创建者标记为离线。

**{选择接管者}**

2.系统选定一名管理员为“临时放映厅创建者”（例如：最早被委任的管理员；若并列则按加入时间顺序）。

**{广播并继续}**

3.系统向全员广播系统提示“放映厅创建者已断线，控制权已由管理员接管”，并标注接管者身份。

4.系统允许该管理员执行所有放映控制与秩序维护操作。

5.用例恢复到基本流的**{放映中控制与全员强制同步（循环）}**处，继续执行步骤 25。

**A10.1 无管理员可接管**

在 A10 放映厅创建者断线（管理员接管）**{选择接管者}**处，若无管理员可接管，则

1.系统向全员提示“放映厅创建者已断线，当前无人可控制放映”。

2.系统保持 VideoState 不变（维持最后一次确认状态）。

3.观众可选择离开房间（该分支结束），或等待放映厅创建者重连（若重连成功则用例恢复到基本流的**{放映中控制与全员强制同步（循环）}**块，继续执行步骤 25）。

**A11 创建房间失败**

在基本流**{创建线上观影室（初始化房间）}**处，若无法创建房间，则

1.系统拒绝创建请求并提示失败原因。

2.若为参数不合法，放映厅创建者可修改输入。

3.用例恢复到基本流的**{启动活动}**块，继续执行步骤 2。

4.若为系统资源不足，系统提示“服务暂时不可用，请稍后重试”。

5.用例终止。

**A12 视频库为空或者检索无结果**

在基本流**{设置片源（检索/选择/加载）}**处，若视频库内无可用片源，则

1.系统提示放映厅创建者“视频库暂无可用片源”。

2.放映厅创建者取消本次放映活动。

3.用例终止

**A13 互动消息发送失败（重试）**

在基本流**{观众讨论互动（并行持续）}**处，若系统分发消息时发现部分观众不可达，则

1.系统对失败的发送执行有限次数重试（例如 3 次）。

2.若仍失败，系统将失败结果记录为“未送达”。

3.系统对发送者提示“部分成员可能未收到消息”。

4.用例恢复到基本流的**{观众讨论互动（并行持续）}**处，用例继续。

**A14 禁言到期自动恢复**

在 A6.2 禁言/解除禁言步骤 2 后，若已达到设置的禁言时长或者 A6.2 步骤 1 选择解除禁言后，则

1.系统自动解除该观众禁言状态。

2.系统可选地通知该观众“禁言已解除”。

3.用例继续。

**A15 过期/乱序同步事件**

在基本流**{观众初始化同步（状态对齐）}**或**{放映中控制与全员强制同步（循环）}**处，若观众收到的 Sync_Event 的操作时间戳早于已应用的最后一次操作，则

1.观众忽略该过期事件。

2.观众保持当前播放状态不变。

3.用例继续。

**A16 视频源切换**

在**{放映中控制与全员强制同步（循环）}**处，放映厅创建者请求更换当前播放内容，则

**{选择新片源}**

1.放映厅创建者选择“更换片源”。

2.系统提示放映厅创建者选择新视频。

3.放映厅创建者从视频库选择一个视频资产并确认。

4.系统验证所选视频可播放。

**{重置并广播}**

5.系统更新 VideoState：

- 切换当前视频源
- 将进度设置为 0
- 将播放状态设置为暂停
- 将倍速设置为 1.0（默认）

6.系统向房间内所有观众发布 Sync_Event(CHANGE_SOURCE)。

7.观众端加载新视频并将进度置为 0，显示“等待放映厅创建者开始播放”。

用例恢复到基本流步骤 20（点击“开始播放”），继续执行步骤 24。

### 1.5. 子流

**S1：验证加入条件**

1.系统检查房间是否存在且处于可加入状态。

2.系统检查房间当前人数是否已达上限。

3.系统检查该观众是否在黑名单/被踢出冷却期内。

4.若房间已加锁：系统验证加入凭据（密码）。

5.若任一步失败，触发对应扩展（E1a/E7 等）；否则返回调用点。

**S2：初始化并应用视频状态（观众侧）**

1.系统读取 VideoState 并向观众返回“当前放映初始化信息”（视频源、字幕、进度、倍速、播放状态等）。

2.观众据此将本地播放器调整到与 VideoState 一致的状态。

3.观众在本地应用个性化偏好（音量/清晰度/弹幕开关），且不影响他人。

4.观众进入同步监听状态。

**S3：处理同步事件（观众侧）**

1.观众接收 Sync_Event。

2.观众根据事件类型更新本地播放状态：PLAY/PAUSE/SEEK/CHANGE_RATE/ CHANGE_SUBTITLE/CHANGE_SOURCE。

3.若事件涉及时间校准：当本地进度与目标进度的偏差超过同步阈值，观众必须执行强制校准。

**S4：验证并广播消息（聊天/弹幕）**

1.系统检查发送者是否有发言权限（未被禁言）。

2.系统校验内容长度与格式（例如：文本 <= 200 字；其他类型按产品规则）。

3.系统接受消息并将其分发给房间内在线成员。

4.接收方依据本地弹幕开关决定是否以弹幕方式展示。

## 2. UC-02 维护视频库（Maintain video library）

### 2.1. 参与者

视频库维护者

### 2.2. 前置条件

• 视频库管理员已成功登录系统，且账户已被识别并授权具备 “维护视频库” 的全部操作权限。

• 系统拥有充足的可用存储空间，用于接收新增视频文件及存储其处理后的衍生资源。

### 2.3. 基本流

**{新增视频 }**

1.视频库管理员进入“视频库管理”主界面，发起‘新增视频’的流程。

2.系统展示一个包含表单和上传区域的新增页面。

3.视频库管理员执行以下操作：

a. 上传视频文件：通过拖拽到指定区域或点击选择文件的方式，选择一个或多个视频文件。系统显示每个文件的上传进度、速度和预估剩余时间。

b. 填写视频信息：为每个视频填写 必要信息（视频名称、分类），以及 可选信息（简介、标签、导演、上映年份、自定义封面图等）。

4.系统在后台顺序执行：

a. 校验当前管理员具备操作权限。

b. 校验表单信息的完整性与合法性（如名称非空且长度合规，分类为预设值）。

c. 校验视频文件（格式支持、大小未超限、文件无损坏）。

5.系统开始接收并持久化存储视频原始文件。

6.系统为每个视频创建一个初始资产记录，并将其状态标记为 “上传中”，成功后转为“处理中”。

7.系统自动启动视频处理流程（如转码、生成预览图），使其满足各类播放终端的兼容性要求。

8.处理过程中，管理员可在视频列表中查看实时状态（如“处理中-50%”）。

9.视频处理成功：系统将视频状态更新为 “可用”，并使其立即可被放映厅搜索和选用。系统可向管理员发送一条站内通知，提示“视频《XXX》已就绪”。

10.管理员在列表中看到新增视频，状态为“可用”。

**{查找与筛选视频 }**

11.视频库管理员在列表页顶部找到搜索与筛选区域。

12.管理员可组合使用以下条件进行查询：

- **关键词搜索：**在“名称/简介”搜索框中输入文字，支持模糊匹配。
- **分类/标签筛选：**通过下拉菜单选择一个或多个预设分类或标签。
- **状态筛选：**筛选指定状态（如“可用”、“处理中”、“处理失败”）的视频。
- **时间范围筛选：**按上传时间或最后修改时间进行范围筛选。

13.管理员提交查询请求。

14.系统执行检索，并返回一个匹配的视频列表。列表支持分页展示，默认按上传时间倒序排列。每条结果展示：封面图、名称、分类、时长、状态、上传时间、最近使用时间。

15.若结果数量较多，管理员可点击表头对列表进行排序（如按名称、时长、使用次数）。

**{编辑视频信息 }**

16.“视频库管理员从列表中找到目标视频，请求对其进行编辑。”

17.系统跳转至该视频的编辑页面，并预加载所有当前信息。

18.管理员修改一个或多个字段（如名称、简介、分类、标签）。

19.管理员点击 “提交”。

20.系统校验修改后的信息合法性。

21.系统保存更新，并刷新视频的“最后修改时间”。

22.系统提示 “信息更新成功”，并自动返回视频列表页，列表中该视频的信息已更新。

**{删除/下架视频 }**

23.“视频库管理员在列表中选择一个或多个目标视频，发起‘删除’操作。”

24.“系统展示操作影响提示并请求最终确认，内容至少包含：

・即将操作的视频名称列表。

・警告提示：“删除后，放映厅将无法再选用此视频。”

・若视频有使用记录，则显示：“该视频已被 [X] 个放映厅使用过。”

・提供 “移至回收站（30 天内可恢复）” 和 “彻底删除” 两个选项（默认为“移至回收站”）。

25.管理员确认操作选项，并点击 “确认删除”。

26.系统后台检查业务约束（例如，是否有视频正在被某个活跃的放映厅使用中）。

・若未被占用：执行删除操作。若选择“移至回收站”，视频状态变为 “已删除” 并从主列表 消失；若选择“彻底删除”，则从存储中移除。

・若被占用：系统中止操作，并明确提示：“操作失败，视频《XXX》正在被放映厅 [房间名] 使用，请稍后再试。”

27.操作成功后，系统提示结果，并自动刷新视频列表。

### 2.4. 备选流

**A1. 输入信息不完整或不合法**

1.在基本流步骤 3 或 5 中，系统检测到管理员提交的视频信息不完整（如名称为空）或非法（如名称超过长度限制）。

2.系统拒绝保存更改，并高亮显示所有存在问题的字段，附上相应错误提示（如“名称为必填项”）。

3.流程返回基本流步骤 2（新增表单页）或步骤 15（编辑页），等待管理员修正信息。

**A2. 视频上传中断或失败**

1.在基本流步骤 3a 中，因网络中断等原因导致视频文件上传失败。

2.系统暂停上传任务，在界面上显示“上传中断”提示，并提供“重试”选项。

3.管理员可选择“重试”，流程返回基本流步骤 3a；或选择“取消”，则用例结束（失败）。

**A3. 存储空间不足**

1.在基本流步骤 4c 或 7 中，系统检测到剩余存储空间不足以接收或处理新视频文件。

2.系统拒绝接收数据，并终止当前上传或处理任务。

3.系统向管理员报告“存储空间不足，请联系系统管理员”错误。

4.流程返回基本流步骤 1（视频库主界面），用例暂时结束。

**A4. 文件格式不合规**

1.在基本流步骤 4c 中，系统检测到视频文件的扩展名不在支持列表内，或实际编码格式与扩展名不符。

2.系统终止处理流程，不保存任何已上传的数据。

3.系统记录安全警告日志，并向管理员返回“不支持的文件格式，请上传 MP4、MOV 等格式”错误提示。

4.用例结束（失败）。

**A5. 文件完整性校验失败**

1.在基本流步骤 4c 中，系统通过校验和（如 MD5）发现接收到的文件与原始文件不一致（可能因网络传输损坏）。

2.系统删除已接收的部分文件。

3.系统向管理员报告“文件传输过程中损坏，请重新上传”错误。

4.流程返回基本流步骤 3a，等待管理员重新上传。

**A6. 视频处理失败**

1.在基本流步骤 9 中，系统在转码或生成预览图等处理过程中发生错误。

2.系统将该视频状态更新为“处理失败”，并记录详细错误日志。

3.系统向管理员显示处理失败状态，并提供“查看错误详情”和“重新处理”选项。

4.管理员可选择重新处理，流程返回基本流步骤 9；或选择重新上传，流程返回基本流步骤 2。

**A7. 查找结果为空**

1.在基本流步骤 14 中，系统执行检索后未发现任何符合当前筛选条件的视频。

2.系统返回一个空列表，并显示提示信息：“未找到符合条件的视频”。

3.系统提供一个“清空筛选条件”的快捷链接。

4.管理员可调整筛选条件后重新查询，流程返回基本流步骤 11；或结束查找。

A8. 编辑信息校验失败

1.在基本流步骤 20 中，系统校验管理员提交的编辑信息时发现非法内容（如名称重复、非法字符）。

2.系统拒绝保存更新，并在相应字段下显示具体错误信息（如“该名称已存在”）。

3.流程返回基本流步骤 16（编辑页面），等待管理员修正信息。

**A9. 删除/下架被拒绝（视频被占用）**

1.在基本流步骤 26 中，系统检查业务约束时发现目标视频正在被一个或多个活跃放映厅使用。

2.系统终止删除/下架操作。

3.系统向管理员报告具体原因，如：“操作失败，视频《XXX》正在‘周五电影夜’房间播放，无法删除”。

4.流程返回基本流步骤 23，管理员可选择其他视频操作或结束用例。

**A10. 批量操作部分成功**

1.在基本流步骤 27 中，管理员执行的批量删除操作中，部分视频因被占用而无法删除。

2.系统完成所有允许的操作后，向管理员返回汇总结果：“成功删除 X 个视频，Y 个视频因被占用而跳过”。

3.系统提供可下载的详细报告，列出所有失败项及其原因。

4.流程返回基本流步骤 23（视频列表页），管理员可针对失败的个别视频进行后续处理。

### 2.5. 后置条件

· **新增成功：**一个新视频资产被创建，经过系统处理后状态变为 “可用”，可被任何放映厅选用。

· **编辑成功：**目标视频的信息被更新，所有变更在后续查询、列表及详情页中立即生效。

· **查找成功：**系统返回一个与查询条件匹配的视频列表（可能为空），此次操作不改变任何核心数据。

· **删除/下架成功：**目标视频被逻辑删除（移至回收站）或标记为 “已下架”，后续不再出现在可选用视频列表中。

# 第4章需求分析

第一次迭代我们选取核心用例“参与社交影院”的基本流，主要实现从创建观影室、设置并播放外部视频链接、观众加入房间、同步观看、聊天、退出和结束放映这一周期涉及到的核心活动。

## 4.1. 健壮性分析

运用 BCE 模式 对社交影院的基本流进行初步解构：通过识别边界类（Boundary）明确人机交互的界面边界，通过控制类（Control）封装复杂的业务规则与流程调度，通过实体类（Entity）承载系统核心的数据状态。

### 1. 边界类

| 类名 | 职责 | 出处 |
|---|---|---|
| RoomCreationUI | 1. 接收创建者填写的初始信息（名称、上限、密码、公告）<br>2. 提供"发起社交影院放映"入口 | "系统提示放映厅创建者填写房间初始信息"；"放映厅创建者选择'发起社交影院放映" |
| RoomManagementUI | 1.展示房间界面及管理功能<br>2.提供设置片源、开始/暂停播放、跳转、设置倍速的入口<br>提供解散房间操作入口 | "系统向放映厅创建者展示房间界面以及房间管理面板"；"放映厅创建者在房间内选择'设置片源'等" |
| RoomJoiningUI | 1.接收加入请求和昵称输入<br>2. 处理身份验证提示 | "观众通过列表/房间号发起加入请求，输入昵称"；"系统对观众执行身份校验" |
| VideoPlayerUI | 1. 展示播放界面，接收同步指令<br>2. 调节本地播放属性（音量、全屏等） | "观众客户端加载资源并将进度设置到 VideoState 指定点"；"观众客户端应用本地属性" |
| VideoSourceSettingUI | 接收创建者输入的外部视频链接 | "放映厅创建者在房间内选择'设置片源'，并输入外部视频链接" |
| ChatUI | 1. 接收观众发送的消息或弹幕<br>2. 展示接收到的聊天消息 | "任一观众可发送聊天消息或弹幕"；"系统将消息广播，并在聊天区展示" |
| RoomDismissalUI | 提示解散房间的不可恢复后果并请求确认 | "系统提示不可恢复的后果并请求确认" |

### 2. 控制类

| 类名 | 职责 | 出处 |
| --- | --- | --- |
| RoomCreationController | 1. 校验创建参数合法性<br>2. 创建观影室并生成唯一标识<br>3. 设置创建者身份和权限 | "系统校验参数合法性"；"系统创建观影室，生成唯一标识"；"设置该昵称由为创建者" |
| VideoStateController | 1. 初始化 VideoState 对象<br>2. 更新 VideoState 为唯一真实数据源<br>3.维护播放状态转换（如：设置为'播放中'）<br>4.验证外部链接格式及可达性<br>5.更换视频源并通知创建者 | "系统初始化视频状态对象"；"系统更新 VideoState，作为新的'唯一真实数据源'"<br>"系统验证该链接格式及可达性"；"系统更换当前视频源，通知创建者已就绪" |
| SyncEventController | 1. 生成同步事件(Sync_Event)<br>2. 向房间内所有观众广播事件 | "系统生成 Sync_Event(PLAY)并广播"；"向房间内所有观众发布 Sync_Event" |
| JoinController | 1. 建立实时连接并加入在线列表<br>2. 广播"观众加入"通知 | "为观众建立实时连接，将其加入在线成员列表"；"系统向成员广播'观众加入'通知" |
| SyncController | 1.读取 VideoState 并向下发初始化数据<br>2.处理 SyncEvent 并更新观众端播放状态 | "系统读取 VideoState 并向新加入观众下发初始化数据" |
| PlaybackController | 1. 验证创建者权限与状态<br>2. 验证控制操作的发起者权限及参数有效性 | "系统验证放映厅创建者权限与连接状态"；"验证发起者具有权限，且参数有效" |
| ChatController | 1.验证消息内容合规性<br>2. 将消息广播给在线成员 | "系统验证消息内容合规"；"系统将消息广播给房间内在线成员" |
| RoomDismissalController | 1. 强制断开所有连接<br>2. 销毁房间实例及其 VideoState | "强制断开所有成员连接，停止广播"；"销毁房间实例及其 VideoState" |

3. 实体类

| 类名 | 职责 | 出处 |
| --- | --- | --- |

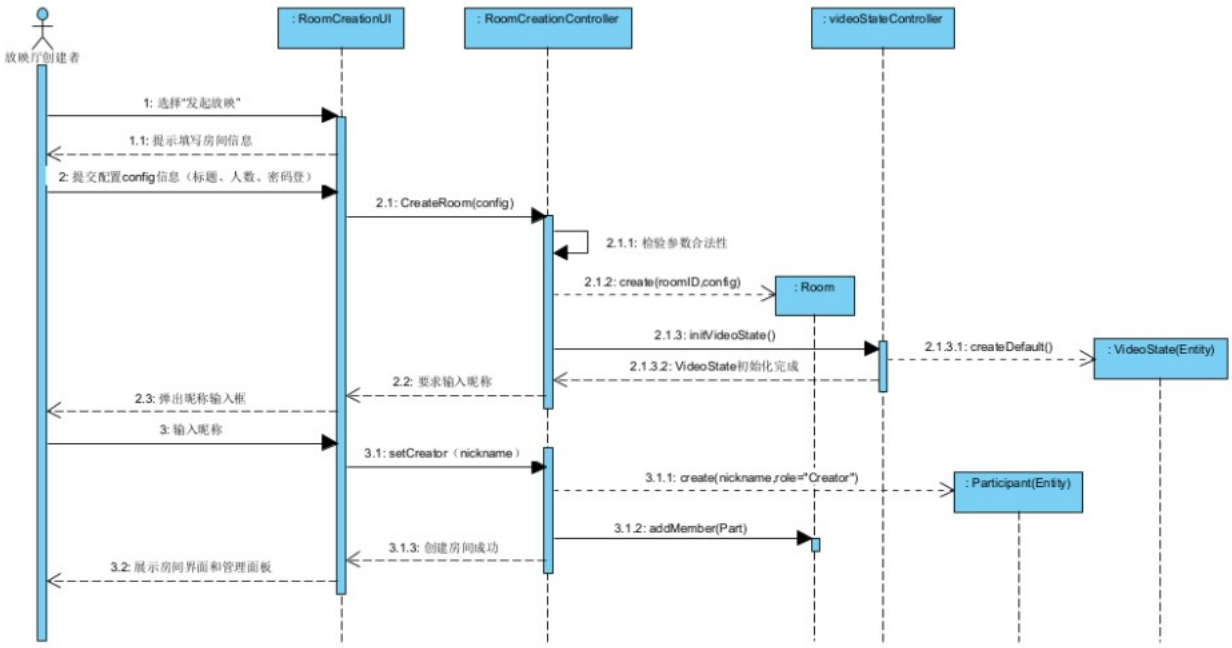
| | | |
|---|---|---|
| Room | 存储房间基本信息（ID、配置、房间内人员等） | "系统创建新的线上观影室，生成唯一标识" |
| VideoState | 存储当前视频源、播放状态、进度、倍速、字幕 | "系统初始化视频状态对象(VideoState)" |
| Participant | 存储成员在该房间的身份（创建者/观众）及通信标识。<br>维护成员在房间内的交互权限（如控制放映、发言）。 | "将其加入房间在线成员列表" |
| Message | 存储聊天消息的内容、发送者、时间等 | "任一观众可发送聊天消息或弹幕" |
| SyncEvent | 封装同步指令（播放/跳转等）及参数 | "系统生成Sync_Event(PLAY)并广播" |

## 4.2. 顺序模型

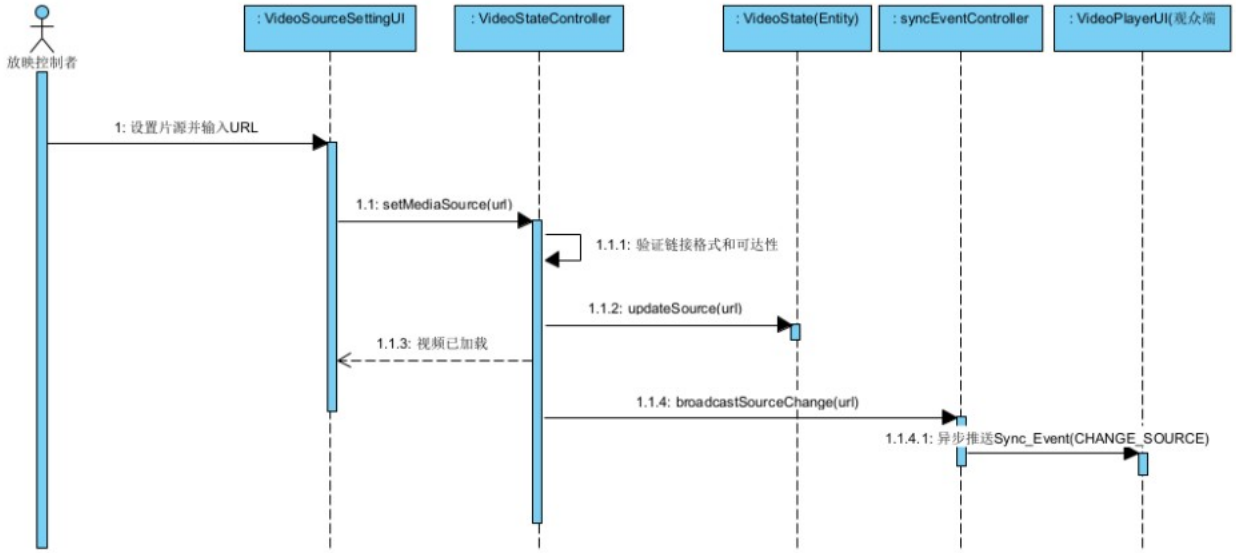
由于基本流跨度较长，我们将基本流涉分成 “创建者 创建-设置片源”、“观众加入-观众初始化”、“放映控制者 同步放映-观众聊天”、“放映室创建者 解散房间” 四个阶段，并绘制顺序图。

### 1. 创建-设置片源 顺序图

**创建放映室并进入**




**设置片源**




**提取的操作和属性：**

**边界类**

职责：负责与用户交互（UI 显示）或外部系统交互，接收输入并展示结果。

**RoomCreationUI (房间创建界面)**

**属性 (Attributes):**

- formFields: Object —— 存储用户输入的标题、密码、最大人数等临时数据。

**操作 (Operations):**

- displayForm(): void —— 展示填写房间信息的表单。
- submitConfig(): void —— (序列图 2) 将用户填写的配置提交给控制器。
- promptNicknameInput(): void —— (序列图 2.3) 弹出弹窗要求输入昵称。
- showRoomInterface(): void —— (序列图 3.2) 创建成功后跳转至房间主界面。

**VideoSourceSettingUI (片源设置界面)**

**操作 (Operations):**

- displayInputBox(): void —— 展示 URL 输入框。
- submitUrl(url): void —— (序列图 1) 用户点击确认后，将 URL 发送给 VideoStateController。
- showError(message): void —— 当链接校验失败时提示用户。

**VideoPlayerUI (播放器界面 - 观众端)**

**操作 (Operations):**

- loadResource(url): void —— 加载视频流。
- updatePlaybackStatus(status): void —— 根据同步事件（播放/暂停）更新本地播放器状态。

控制类

**RoomCreationController (房间创建控制器)**

操作 **(Operations):**

- createRoom(config): void —— 接收 UI 传来的配置，协调创建流程。
- validateConfig(config): Boolean —— (序列图 2.1.1) 校验参数合法性（如密码强度、人数限制）。
- setCreator(nickname): void —— (序列图 3.1) 将当前用户关联为房间的创建者（Owner）。

**VideoStateController (视频状态控制器)**

操作 **(Operations):**

- initVideoState(): void —— (序列图 2.1.3) 在房间创建时，初始化视频状态实体。
- setMediaSource(url): void —— (序列图 1.1) 响应设置片源的请求。
- validateUrl(url): Boolean —— (序列图 1.1.1) 验证视频链接的格式和可达性（是否有效）。
- broadcastSourceChange(url): void —— (序列图 1.1.4) 通知同步控制器去广播“切片”事件。

**SyncEventController (同步事件控制器)**

操作 **(Operations):**

- generateSyncEvent(type, data): Event —— 封装同步指令（如播放、暂停、切源）。
- broadcastToRoom(event): void —— (序列图 1.1.4.1) 将事件推送到房间内所有在线观众的客户端。

实体类

**Room (房间)**

属性 **(Attributes):**

- roomID: String —— 房间的唯一标识符。
- config: Object —— 包含房间标题、人数上限、密码、公告等配置信息。
- creatorID: String —— 创建者的 ID，用于权限判断。
- participantList: List<Participant> —— 当前房间内所有在线成员的列表。

**操作 (Operations):**

- create(roomID, config): Constructor —— 根据配置信息初始化一个房间实例。
- addMember(participant): void —— 将一个新的参与者对象加入到成员列表中。
- removeMember(participantID): void —— 从成员列表中移除离开的用户。
- dismiss(): void —— 标记房间为解散状态，清理资源。

**VideoState (视频状态)**

**属性 (Attributes):**

- videoURL: String —— 当前播放的视频资源链接。
- isPlaying: Boolean —— 播放状态（true 为播放，false 为暂停）。
- currentTime: Double —— 当前视频播放的进度时间戳。
- playbackSpeed: Float —— 当前的播放倍速（如 1.0, 1.5）。
- subtitleSettings: Object —— 字幕相关的配置。

**操作 (Operations):**

- createDefault(): void —— 初始化默认状态（无视频、暂停、进度 0）。
- updateSource(url): void —— 更新视频源地址，重置进度。
- updateState(isPlaying, time): void —— 更新播放/暂停状态和当前进度。

**Participant (参与者)**

**属性 (Attributes):**

- nickname: String —— 用户在房间内显示的昵称。
- role: Enum —— 用户的角色（"Creator" 创建者 / "Audience" 观众）。
- connectionID: String —— 用户的网络连接标识（Socket ID 等）。
- permissions: List<String> —— 用户拥有的权限列表（如控制播放、发言）。

**操作 (Operations):**

- create(nickname, role): Constructor —— 创建一个参与者实例并设定初始角色。
- hasPermission(action): Boolean —— 检查该用户是否拥有执行某动作的权限。

**SyncEvent (同步事件)**

**属性(Attributes):**

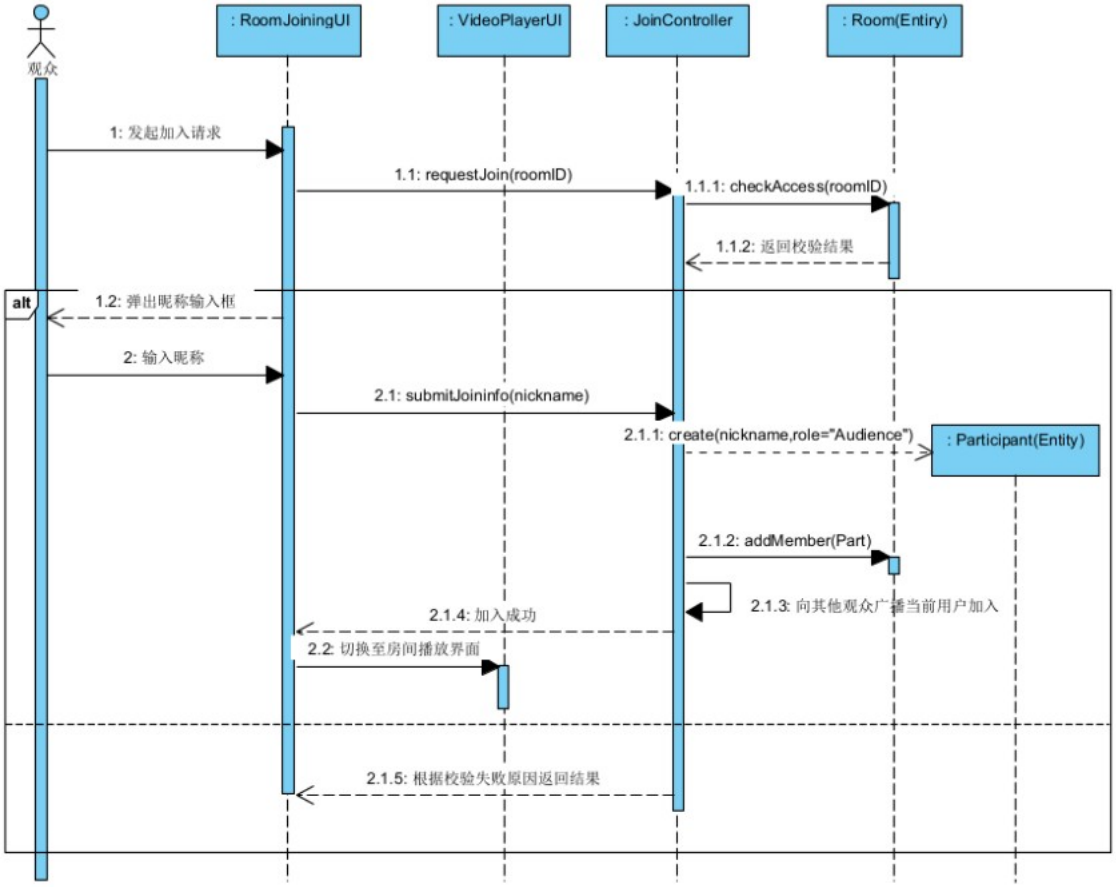
- eventType (事件类型: PLAY, PAUSE, SEEK, CHANGE_SOURCE), payload (负载数据: 时间戳, URL)。

**操作(Operations):**

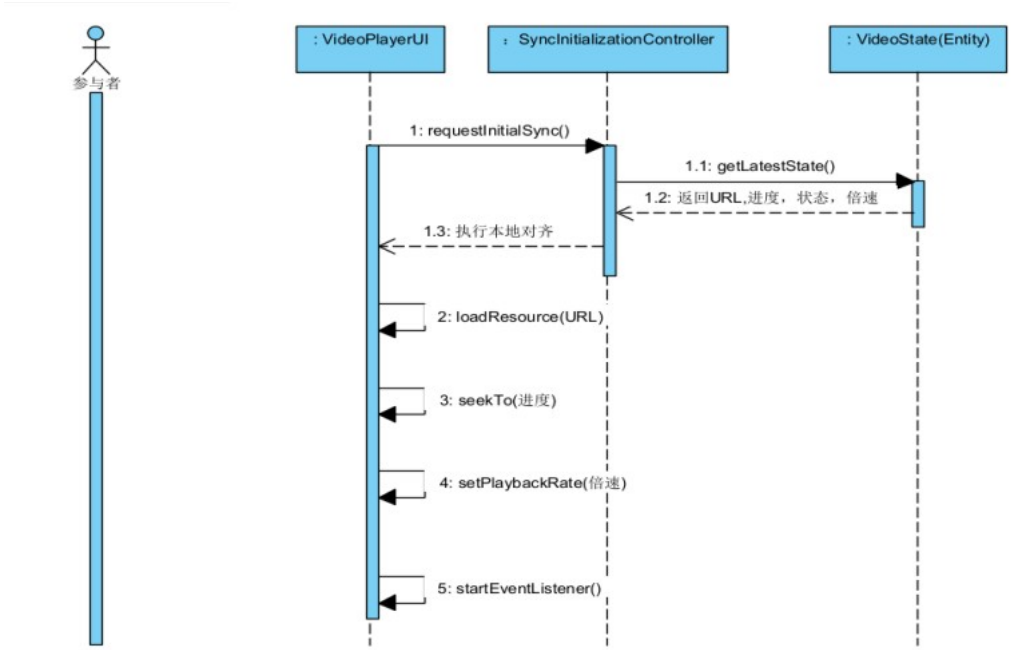
・serialize() (序列化为网络传输格式)。

2. 观众加入-观众初始化 顺序图

观众加入房间并建立连接 顺序图




观众加入房间后初始化本地播放器顺序图




**提取的操作和属性：**

**边界类**

**RoomJoiningUI (房间加入界面)**

**操作 (Operations):**

- promptNicknameInput(): void —— 1.2 当房间校验通过后，显示输入框让用户填写昵称。
- switchToRoomInterface(): void —— 2.2 登录成功后，关闭当前窗口，跳转/加载到 VideoPlayerUI 界面。
- showError(reason): void —— 2.1.5 如果校验失败（如房间不存在），向用户展示错误提示。

**VideoPlayerUI (播放器界面)**

**属性 (Attributes):**

- playerInstance: Object —— 本地播放器引擎实例（如 HTML5 Video Element，本项目采用 artPlayer）。

**操作 (Operations):**

- loadResource(URL): void —— 2 根据同步到的 URL，加载视频流。
- seekTo(time): void —— 3 将进度条拖动到服务器当前的播放时间点，实现进度同步。
- setPlaybackRate(speed): void —— 4 设置播放倍速（如 1.5 倍速）。
- startEventListener(): void—— 5 初始化完成后，开始监听后续的 WebSocket 消息（如暂停、跳转等实时指令），以保持持续同步。

控制类

**JoinController (加入流程控制器)**

**操作 (Operations):**

- requestJoin(roomID): void——1.1 接收 UI 发起的加入请求，开始后续的校验流程。
- checkAccess(roomID): Boolean——1.1.1 调用实体类校验房间号是否有效。
- submitJoinInfo(nickname): void——2.1 接收用户输入的昵称，继续加入流程。
- broadcastMemberJoin(participant): void——2.1.3 (内部逻辑) 向房间内已有的其他观众广播“新用户加入”的消息，以便其他人更新成员列表。

**SyncController (同步执行器)**

**操作 (Operations):**

- requestInitialSync(): void——1 当观众进入播放界面后，立即被调用。用于拉取服务器端的当前播放状态。
- returnSyncData(url, time, state, speed): void——1.3 将从实体类获取的数据打包，返回给前端 UI 进行“本地对齐”。

实体类

**Room (房间)**

**操作 (Operations):**

- checkAccess(roomID): Boolean——1.1.1 检查房间是否存在，以及检查房间状态（如是否已满员、是否已解散）。虽然图中只传了 ID，但逻辑上包含了对房间可用性的校验。
- addMember(participant): void——2.1.2 将新创建的 Participant 对象加入到该房间的当前成员列表中，确保持久化数据更新。

**Participant (参与者)**

**操作 (Operations):**

- create(nickname, role="Audience"): Constructor——2.1.1 构造函数。实例化一个新的参与者对象，并将其角色硬编码或默认为“Audience”（观众），区别于创建者。

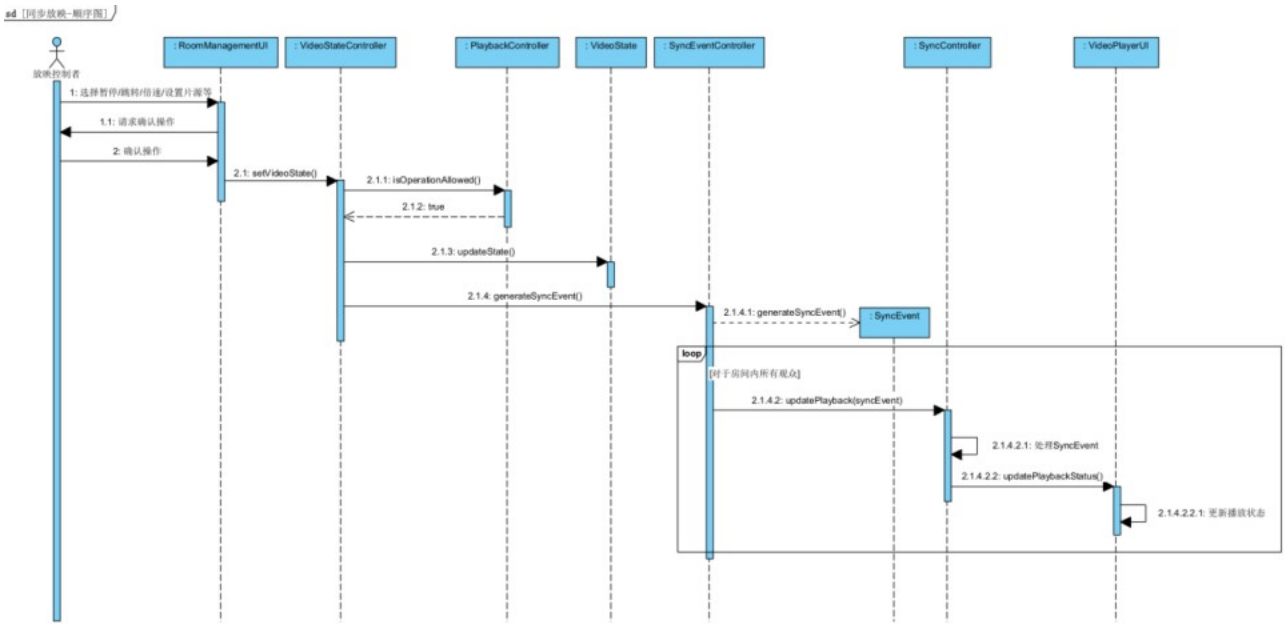
**VideoState (视频状态)**

**操作 (Operations):**

- getLatestState(): Object——1.1 获取当前房间内“事实来源（Source of Truth）”的视频状态。 返回数据：包含 videoURL (视频地址), currentTime (当前进度), isPlaying (播放状态), playbackSpeed (倍速)。

3. 同步放映-聊天 顺序图

修改播放状态并同步




提取的操作和属性：

边界类

职责：负责与用户交互（UI 显示）或外部系统交互，接收输入并展示结果。

**RoomManagementUI (房间管理界面)**

**操作 (Operations):**

- play(): void —— 放映控制者选择播放视频。
- pause(): void —— 放映控制者选择暂停视频。
- jumpTo(): void —— 放映控制者选择跳转播放进度到指定时间。
- setSpeed(): void —— 放映控制者选择设置播放倍速。
- setMediaSource(): void —— 放映控制者选择设置视频源。
- confirm(): void —— 放映控制者选择确认操作。
- cencel(): void —— 放映控制者选择取消操作。

**VideoPlayerUI (播放器界面 – 观众端)**

**操作 (Operations):**

- updatePlaybackStatus(status): void —— 根据指令更新本地播放器状态。

控制类

**VideoStateController (视频状态控制器)**

**操作 (Operations):**

・play(): void —— 播放视频。

・pause(): void —— 暂停视频。

・jumpTo(): void —— 跳转播放进度到指定时间。

・setSpeed(): void —— 设置播放倍速。

・setMediaSource(): void —— 设置视频源。

**PlaybackController (视频状态控制器)**

**操作 (Operations):**

・isOperationAllowed(): bool —— 返回控制操作的发起者是否有权限及参数是否有效。

**SyncEventController (同步事件控制器)**

**操作 (Operations):**

・generateSyncEvent(type, data): Event —— 封装同步指令（如播放、暂停、切源）。

**SyncController (同步执行器)**

**操作 (Operations):**

・updatePlayback(syncEvent): void —— 解析同步指令为本地播放器可识别的一系列指令。

实体类

VideoState (视频状态)

**属性 (Attributes):**

・videoURL: String —— 当前播放的视频资源链接。

・isPlaying: Boolean —— 播放状态（true 为播放，false 为暂停）。

・currentTime: Double —— 当前视频播放的进度时间戳。

・playbackSpeed: Float —— 当前的播放倍速。

**操作 (Operations):**

・updateSource(url): void —— 更新视频源地址，重置进度。

・updateState(isPlaying, time): void —— 更新播放/暂停状态和当前进度。

**SyncEvent (同步事件)**

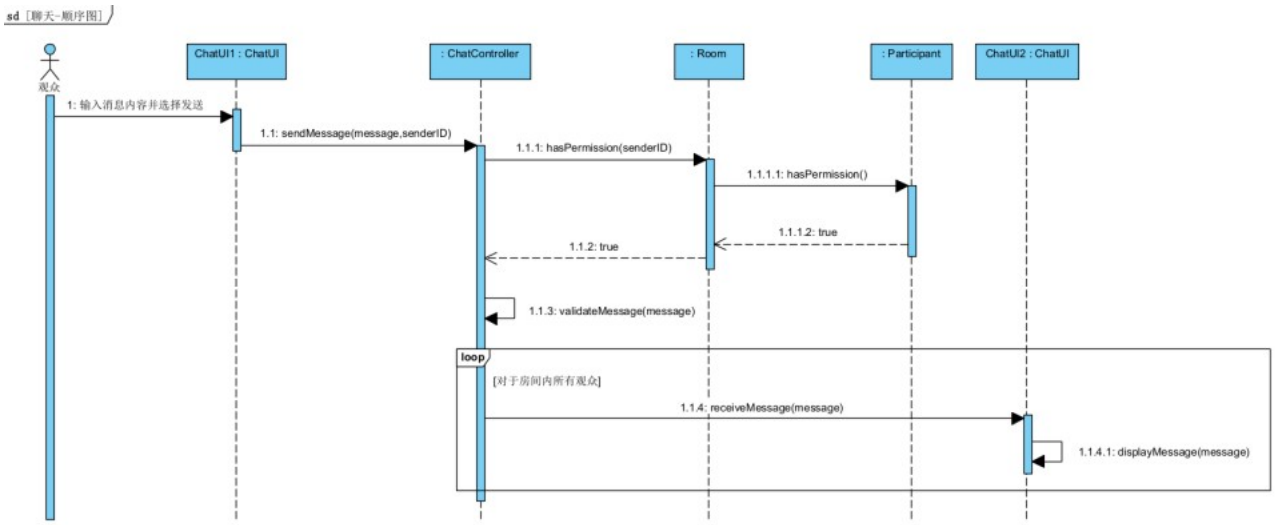
**属性(Attributes):**

・eventType (事件类型: PLAY, PAUSE, SEEK, CHANGE_SOURCE), payload (负载数据: 时间戳, URL)。

**操作(Operations):**

・serialize() (序列化为网络传输格式)。

**发送聊天消息并同步**




**边界类**

**ChatUI (房间聊天界面)**

**操作 (Operations):**

・getMessage(): void —— 获取观众要发送的消息内容。

・sendMessage(): void —— 观众选择发送消息。

・receiveMessage(): void —— 接收消息。

・displayMessage(): void —— 在聊天界面渲染消息。

**控制类**

**ChatController (聊天消息控制器)**

**操作 (Operations):**

・sendMessage(message): void —— 请求广播消息。

・validateMessage(message): void —— 检查消息内容合规性。

**实体类**

**Room (房间)**

**属性 (Attributes):**

・participantsList：List<Particicpant>——成员列表

**操作 (Operations):**

・hasPermission(id)：bool —— 查询成员在房间内的交互权限

**Participant (参与者)**

**属性 (Attributes):**

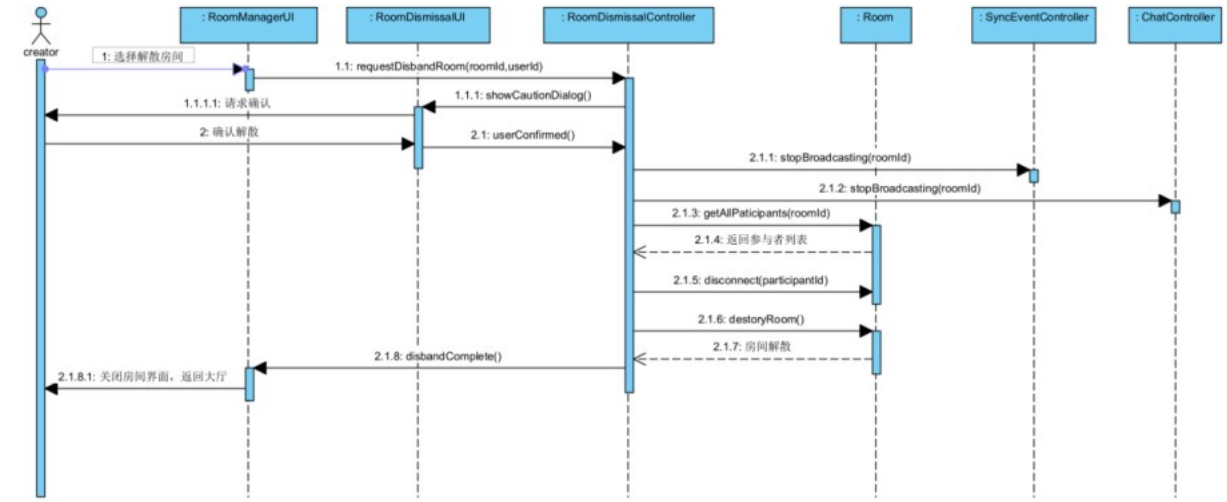
・connectionID: String —— 用户的网络连接标识（Socket ID 等）。

・permissions: List<String> —— 用户拥有的权限列表（如控制播放、发言）。

**操作 (Operations):**

・hasPermission(action): bool —— 检查该用户是否拥有执行某动作的权限。

4. 解散房间 顺序图




**提取的操作和属性：**

**边界类**

**RoomManagementUI (房间管理界面)**

**操作 (Operations):**

・dismissRoom(): void——1 向放映厅创建者提供解散房间的选项。

・requestDismissRoom(roomId, userId): void —— 1.1 当放映厅创建者选择解散房间时向 RoomDismissalController 发出解散房间的请求。

・colse(): void——2.1.8.1 解散房间后，关闭房间管理界面。

**RoomDismissalUI (解散确认界面)**

**属性 (Attributes):**

・caution: string——向放映厅创建者发出的解散房间警示信息。

**操作 (Operations):**

・showCautionDialog(): void——1.1.1 展示解散房间确认对话框

・userConfirmed(): void——2.1 放映厅创建者确认解散房间，向 RoomDismissalController 传递信息。

・close(): void——2 放映厅创建者确认房间后，关闭解散确认界面。

控制类

**RoomDismissalController (房间解散控制器)**

**操作 (Operations):**

・acceptDismissRequest():void——接受解散房间的请求，并且展示 RoomDismissalUI。

・getAllParticipants(roomId): List<Participant>——2.1.3 获取该房间内观众的信息。

・disconnect(participantId): void——2.1.5 断开与房间内所有观众的连接。

・destroyRoom(roomId): void——2.1.6 解散房间

・dismissComplete(): void——2.1.8 成功解散房间，并且关闭 RoomManagementUI。

**SyncEventController (同步事件控制器)**

**操作 (Operations):**

・stopBroadcasting(roomId): void——2.1.1 停止向该房间的同步事件广播服务。

**ChatController(聊天控制器)**

**操作 (Operations):**

・stopBroadcasting(roomId): void——2.1.2 停止向该房间的消息广播服务。

实体类

**Room (房间)**

**属性 (Attributes):**

・participantsList：List<Particicpant>——成员列表

**操作 (Operations):**

・returnParticipants():List<Participant>——2.1.4 返回房间内观众的信息

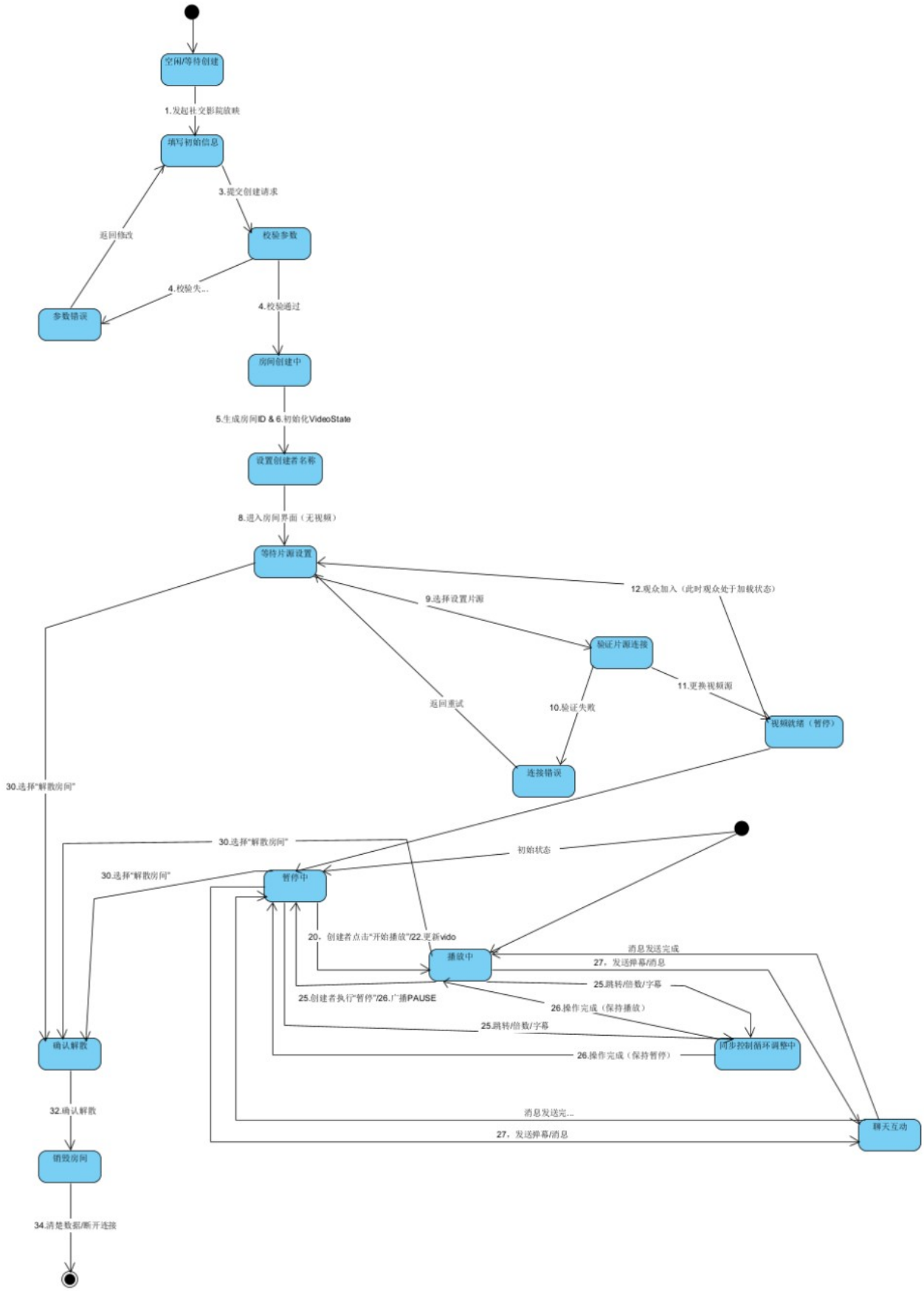
・destroy():void——2.1.6 房间销毁

## 4.3. 状态模型

本板块将针对具有复杂生命周期的核心类，绘制状态机图。我们将重点捕捉状态的迁移触发条件与内部转换逻辑，确保系统在处理诸如“暂停、跳转、缓冲”等高频并发事件时，能够始终保持内在逻辑的一致性，防止由于状态冲突导致的同步失效。

### 1. UML 状态机图

参与社交影院


空闲/等待创建
1.发起社交影院放映
填写初始信息
3.提交创建请求
返回修改
校验参数
4.校验失...
4.校验通过
参数错误
房间创建中
5.生成房间ID & 6.初始化VideoState
设置创建者名称
8.进入房间界面（无视频）
等待片源设置
12.观众加入（此时观众处于加载状态）
9.选择设置片源
验证片源连接
11.更换视频源
返回重试
10.验证失败
视频就绪（暂停）
连接错误
30.选择"解散房间"
30.选择"解散房间"
初始状态
30.选择"解散房间"
暂停中
20、创建者点击"开始播放"/22.更新vido
播放中
消息发送完成
27，发送弹幕/消息
25.跳转/倍数/字幕
25.创建者执行"暂停"/26.广播PAUSE
26.操作完成（保持播放）
25.跳转/倍数/字幕
确认解散
26.操作完成（保持暂停）
同步控制循环调整中
32.确认解散
消息发送完...
27，发送弹幕/消息
聊天互动
销毁房间
34.清楚数据/断开连接

2. **状态模型的核心状态分类**

整个流程的状态可分为 4 个阶段 + 2 个并行子状态，对应图中的模块：

| 阶段分类 | 核心状态（图中节点） | 对应基本流步骤 |
|---|---|---|
| 创建与初始化阶段 | 空闲 / 待创建、填写初始信息、校验参数、房间创建中、设置创建者昵称、等待片源设置 | 步骤 1-8 |
| 片源设置阶段 | 验证片源链接、视频就绪_暂停 | 步骤 9-11 |
| 放映运行阶段 | 暂停中、播放中、同步控制循环 | 步骤 20-26 |
| 结束阶段 | 确认解散、销毁房间 | 步骤 30-35 |
| 并行子状态 1 | 观众加入（同步对齐） | 步骤 12-19 |
| 并行子状态 2 | 聊天互动 | 步骤 27-29 |

3. **状态流转的关键逻辑**

结合图和基本流，核心状态的触发条件与行为如下：

1. **创建与初始化阶段**

・空闲 / 待创建 → 填写初始信息：触发事件是 “放映厅创建者发起社交影院放映”（步骤 1）。

・填写初始信息 → 校验参数：触发事件是 “提交创建请求”（步骤 3）。

・校验参数 → 房间创建中：触发条件是 “参数校验通过”（步骤 4）；行为是 “生成房间 ID + 初始化 VideoState”（步骤 5-6）。

・房间创建中 → 等待片源设置：触发事件是 “设置创建者昵称”（步骤 7）；行为是 “展示房间界面”（步骤 8）。

2. **片源设置阶段**

・等待片源设置 → 验证片源链接：触发事件是 “选择设置片源并输入链接”（步骤 9）。

・验证片源链接 → 视频就绪_暂停：触发条件是 “链接验证通过”（步骤 10）；行为是 “更新 VideoState + 广播 CHANGE_SOURCE”（步骤 11）。

3. **放映运行阶段（核心循环）**

・视频就绪_暂停 → 暂停中：默认进入的初始状态（VideoState 初始为暂停）。

・暂停中 → 播放中：触发事件是 “创建者点击开始播放”（步骤 20）；行为是 “更新 VideoState 为播放 + 广播 PLAY”（步骤 22-23）。

・播放中 / 暂停中 → 同步控制循环：触发事件是 “创建者执行控制操作（播放 / 暂停 /

跳转 / 调倍速）”（步骤 25）；行为是 “校验权限 → 更新 VideoState → 广播对应 Sync_Event → 观众强制对齐”（步骤 26）。

4. **并行子状态的流转**

・**观众加入：**可在 “等待片源设置 / 暂停中 / 播放中” 任意状态触发（步骤 12）；行为是 “校验身份 → 建立连接 → 下发 VideoState 同步数据 → 观众对齐状态”（步骤 13-19）。

・**聊天互动：**可在 “暂停中 / 播放中” 任意状态触发（步骤 27）；行为是 “验证消息合规 → 广播消息”（步骤 28-29），且不影响视频播放状态。

5. **结束阶段**

・**任意状态 → 确认解散：**触发事件是 “创建者选择解散房间”（步骤 30）。

・**确认解散 → 销毁房间：**触发事件是 “创建者确认解散”（步骤 32）；行为是 “断开连接 + 销毁房间 / VideoState”（步骤 33-34）。

#### 4. 状态模型的核心特点

1.**VideoState 作为唯一数据源：**所有播放状态的变更都以 VideoState 为准，观众端仅被动同步（对应步骤 16、22、26）。

2.**观众加入的同步机制：**模型中体现了观众加入不是直接进入播放，而是先进入一个隐式的 “同步对齐” 过程（步骤 16-17）。这解释了为什么新观众加入时能看到正确的进度，而不是从头开始。

3.**片源的独立性：**设计了 STATE_ROOM_READY_NO_VIDEO 状态。这意味着房间创建后、视频加载前，房主和观众（如果提前进入）可以处于一个 “只有聊天 / 配置，没有画面” 的状态。

4.**并行行为独立：**“观众加入” “聊天互动” 是并行于播放控制的行为，不影响主状态流转。

5.**强制同步机制：**播放控制操作会触发全局 Sync_Event，观众端必须强制对齐（对应步骤 26）。

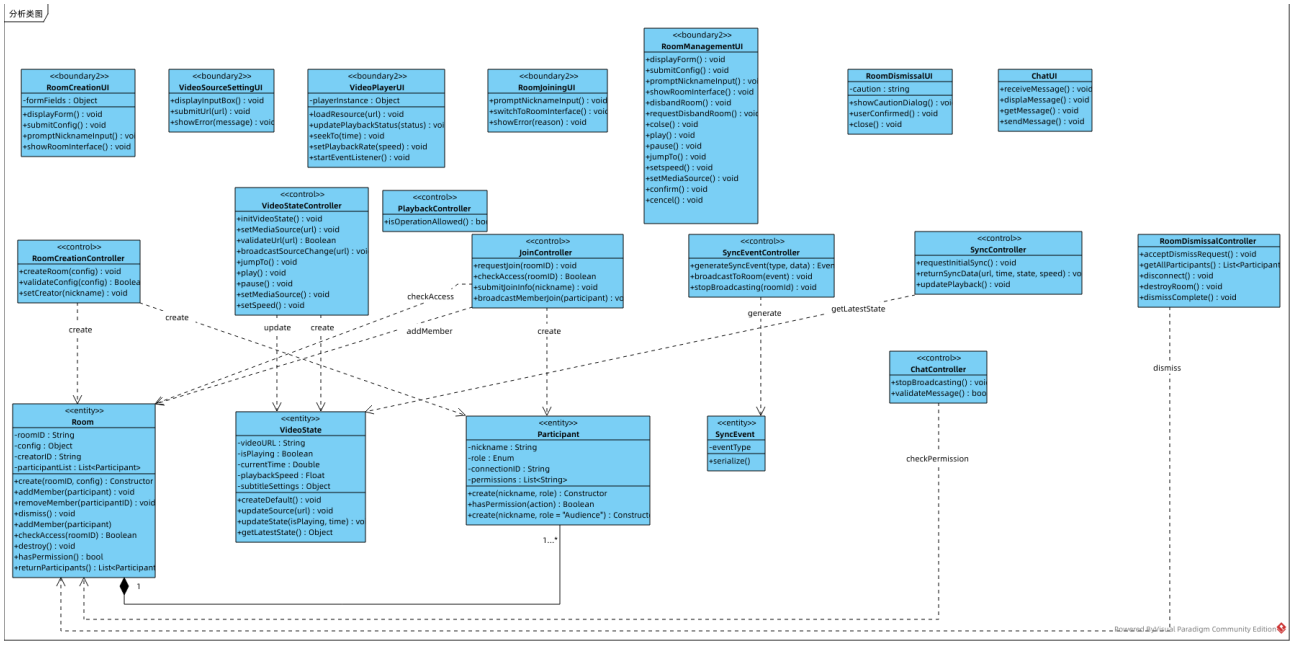
### 4.4. 完善分析类图

在本板块中，我们将执行以下关键工作：

提取操作：根据顺序图中的消息传递，为每个类补齐必备的函数签名。

定义属性：根据状态模型中的状态位和业务需求，明确各实体的核心数据字段。

梳理关联：根据对象间的协作频率与依赖强度，建立严谨的逻辑连接（如聚合、组合关系）。




**操作和属性介绍：**

**边界类**

**RoomCreationUI (房间创建界面)**

**属性 (Attributes):**

- formFields: Object —— 存储用户输入的标题、密码、最大人数等临时数据。

**操作 (Operations):**

- displayForm(): void —— 展示填写房间信息的表单。
- submitConfig(): void —— 将用户填写的配置提交给控制器。
- promptNicknameInput(): void —— 弹出弹窗要求输入昵称。
- showRoomInterface(): void —— 创建成功后跳转至房间主界面。

**VideoSourceSettingUI (片源设置界面)**

**操作 (Operations):**

- displayInputBox(): void —— 展示 URL 输入框。
- submitUrl(url): void —— 用户点击确认后，将 URL 发送给 VideoStateController。
- showError(): void —— 当链接校验失败时提示用户。

**VideoPlayerUI (播放器界面 - 观众端)**

**属性 (Attributes):**

- playerInstance: Object —— 本地播放器引擎实例（如 HTML5 Video Element，本项目采用 artPlayer）。

**操作 (Operations):**

- loadResource(): void —— 加载视频流。
- updatePlaybackStatus(): void —— 根据同步事件（播放/暂停）更新本地播放器状态。
- seekTo(time): void —— 将进度条拖动到服务器当前的播放时间点，实现进度同步。
- setPlaybackRate(speed): void —— 设置播放倍速（如 1.5 倍速）。
- startEventListener(): void—— 初始化完成后，开始监听后续的 WebSocket 消息（如暂停、跳转等实时指令），以保持持续同步。

**RoomJoiningUI (房间加入界面)**

**操作 (Operations):**

- promptNicknameInput(): void —— 当房间校验通过后，显示输入框让用户填写昵称。
- switchToRoomInterface(): void —— 登录成功后，关闭当前窗口，跳转/加载到 VideoPlayerUI 界面。
- showError(reason): void —— 如果校验失败（如房间不存在），向用户展示错误提示。

**RoomManagementUI (房间管理界面)**

**操作 (Operations):**

- play(): void —— 放映控制者选择播放视频。
- pause(): void —— 放映控制者选择暂停视频。
- jumpTo(): void —— 放映控制者选择跳转播放进度到指定时间。
- setSpeed(): void —— 放映控制者选择设置播放倍速。
- setMediaSource(): void —— 放映控制者选择设置视频源。
- confirm(): void —— 放映控制者选择确认操作。
- cencel(): void —— 放映控制者选择取消操作。
- dismissRoom(): void——向放映厅创建者提供解散房间的选项。
- requestDismissRoom(roomId, userId): void —— 当放映厅创建者选择解散房间时向 RoomDismissalController 发出解散房间的请求。
- colse(): void——解散房间后，关闭房间管理界面。

**ChatUI (房间聊天界面)**

**操作 (Operations):**

- getMessage(): void —— 获取观众要发送的消息内容。
- sendMessage(): void —— 观众选择发送消息。

- receiveMessage(): void —— 接收消息。
- displayMessage(): void —— 在聊天界面渲染消息。

**RoomDismissalUI (解散确认界面)**

**属性 (Attributes):**

- caution: string——向放映厅创建者发出的解散房间警示信息。

**操作 (Operations):**

- showCautionDialog(): void——展示解散房间确认对话框
- userConfirmed(): void——放映厅创建者确认解散房间，向 RoomDismissalController 传递信息。
- close(): void—— 放映厅创建者确认房间后，关闭解散确认界面。

**控制类**

**RoomCreationController (房间创建控制器)**

**操作 (Operations):**

createRoom(config): void —— 接收 UI 传来的配置，协调创建流程。

validateConfig(config): Boolean —— 校验参数合法性（如密码强度、人数限制）。

setCreator(nickname): void —— 将当前用户关联为房间的创建者（Owner）。

**VideoStateController (视频状态控制器)**

**操作 (Operations):**

- initVideoState(): void —— 在房间创建时，初始化视频状态实体。
- setMediaSource(url): void —— 设置视频源。
- validateUrl(url): Boolean —— 验证视频链接的格式和可达性（是否有效）。
- broadcastSourceChange(url): void —— 通知同步控制器去广播“切片”事件。
- play(): void —— 播放视频。
- pause(): void —— 暂停视频。
- jumpTo(): void —— 跳转播放进度到指定时间。
- setSpeed(): void —— 设置播放倍速。。

**SyncEventController (同步事件控制器)**

**操作 (Operations):**

- generateSyncEvent(type, data): Event —— 封装同步指令（如播放、暂停、切源）。

- broadcastToRoom(event): void —— 将事件推送到房间内所有在线观众的客户端。
- stopBroadcasting(roomId): void——停止向该房间的同步事件广播服务。

**JoinController (加入流程控制器)**

**操作 (Operations):**

- requestJoin(roomID): void——接收 UI 发起的加入请求，开始后续的校验流程。
- checkAccess(roomID): Boolean—— 调用实体类校验房间号是否有效。
- submitJoinInfo(nickname): void—— 接收用户输入的昵称，继续加入流程。
- broadcastMemberJoin(participant): void——(内部逻辑) 向房间内已有的其他观众广播“新用户加入”的消息，以便其他人更新成员列表。

**SyncController (同步执行器)**

**操作 (Operations):**

- requestInitialSync(): void——当观众进入播放界面后，立即被调用。用于拉取服务器端的当前播放状态。
- returnSyncData(url, time, state, speed): void—— 将从实体类获取的数据打包，返回给前端 UI 进行“本地对齐”。
- updatePlayback(syncEvent): void —— 解析同步指令为本地播放器可识别的一系列指令。
- PlaybackController (视频状态控制器)

**操作 (Operations):**

- isOperationAllowed(): bool —— 返回控制操作的发起者是否有权限及参数是否有效。

**ChatController (聊天消息控制器)**

**操作 (Operations):**

- sendMessage(message): void —— 请求广播消息。
- validateMessage(message): void —— 检查消息内容合规性。
- stopBroadcasting(roomId): void——停止向该房间的消息广播服务。

**RoomDismissalController (房间解散控制器)**

**操作 (Operations):**

- acceptDismissRequest():void——接受解散房间的请求，并且展示 RoomDismissalUI。
- getAllParticipants(roomId): List<Participant>——获取该房间内观众的信息。
- disconnect(participantId): void——断开与房间内所有观众的连接。
- destroyRoom(roomId): void—— 解散房间

・dismissComplete(): void——成功解散房间，并且关闭 RoomManagementUI。

实体类

**Room (房间)**

属性 **(Attributes):**

・roomID: String —— 房间的唯一标识符。

・config: Object —— 包含房间标题、人数上限、密码、公告等配置信息。

・creatorID: String —— 创建者的 ID，用于权限判断。

・participantList: List<Participant> —— 当前房间内所有在线成员的列表。

操作 **(Operations):**

・create(roomID, config): Constructor —— 根据配置信息初始化一个房间实例。

・addMember(participant): void —— 将一个新的参与者对象加入到成员列表中。

・removeMember(participantID): void —— 从成员列表中移除离开的用户。

・dismiss(): void —— 标记房间为解散状态，清理资源。

・checkAccess(roomID): Boolean—— 检查房间是否存在，以及检查房间状态（如是否已满员、是否已解散）。虽然图中只传了 ID，但逻辑上包含了对房间可用性的校验。

・addMember(participant): void—— 将新创建的 Participant 对象加入到该房间的当前成员列表中，确保持久化数据更新。

・hasPermission(id)：bool —— 查询成员在房间内的交互权限

・returnParticipants():List<Participant>——返回房间内观众的信息

・destroy():void——房间销毁

**VideoState (视频状态)**

属性 **(Attributes):**

・videoURL: String —— 当前播放的视频资源链接。

・isPlaying: Boolean —— 播放状态（true 为播放，false 为暂停）。

・currentTime: Double —— 当前视频播放的进度时间戳。

・playbackSpeed: Float —— 当前的播放倍速（如 1.0, 1.5）。

・subtitleSettings: Object —— 字幕相关的配置。

操作 **(Operations):**

・createDefault(): void —— 初始化默认状态（无视频、暂停、进度 0）。

・updateSource(url): void —— 更新视频源地址，重置进度。

- updateState(isPlaying, time): void —— 更新播放/暂停状态和当前进度。
- getLatestState(): Object——获取当前房间内“事实来源（Source of Truth）”的视频状态。返回数据：包含 videoURL (视频地址), currentTime (当前进度), isPlaying (播放状态), playbackSpeed (倍速)。

**Participant (参与者)**

**属性 (Attributes):**

- nickname: String —— 用户在房间内显示的昵称。
- role: Enum —— 用户的角色（"Creator" 创建者 / "Audience" 观众）。
- connectionID: String —— 用户的网络连接标识（Socket ID 等）。
- permissions: List<String> —— 用户拥有的权限列表（如控制播放、发言）。

**操作 (Operations):**

- create(nickname, role): Constructor —— 创建一个参与者实例并设定初始角色。
- hasPermission(action): Boolean —— 检查该用户是否拥有执行某动作的权限。

**SyncEvent (同步事件)**

**属性(Attributes):**

- eventType (事件类型: PLAY, PAUSE, SEEK, CHANGE_SOURCE), payload (负载数据: 时间戳, URL)。

**操作(Operations):**

- serialize() (序列化为网络传输格式)。

# 第5章 架构设计

## 5.1. 架构目标与约束

### 1. 架构目标

1.**可维护性与模块化：**通过清晰的模块划分和接口设计，确保系统各部分职责明确，便于独立开发、测试和维护。即使采用单体部署，在逻辑上也要实现"微服务化"的模块分离。

2.**实时性与低延迟：**优化实时通信机制，确保视频同步延迟低于1秒，消息传递延迟低于500毫秒，提供流畅的同步观影体验。

3.**可扩展性与演进能力：**架构设计需支持未来功能扩展，包括用户规模增长、功能模块增加以及可能的微服务拆分，而无需大规模重构。

4.**开发效率与团队协作：**前后端分离架构支持并行开发，明确的接口契约减少团队间依赖，提高开发效率。

5.**技术栈统一与生态整合：**充分利用JavaScript生态系统优势，前后端技术栈保持一致，降低学习成本和技术栈切换开销。

### 2. 架构约束

1.**存储约束（第一次迭代）：**

a.所有业务数据（房间、参与者、视频状态）存储在进程内存中

b.服务器重启或进程崩溃导致数据完全丢失

c.无持久化存储需求，简化部署复杂度

d.用户会话和权限信息临时存储

2.**部署约束：**

a.单体应用部署，所有功能模块打包在一个Node.js进程中

b.单服务器部署，不考虑分布式架构

c.内存存储限制并发用户数和房间数

3.**技术栈约束：**

a.前端：Vue.js 3.x + JavaScript(TypeScript)

b.后端：Node.js v25.2.1 + Express.js + Socket.IO

c.视频处理：FFmpeg + Nginx HLS模块

d.操作系统：Linux（manjaro/Archlinux）OR windows

4.**兼容性约束：**

a.支持现代浏览器（Chrome 90+、Firefox 88+、Safari 14+）

b.响应式设计，支持 PC 端和移动端浏览器访问

## 5.2. 软件架构风格说明

### 1. 前后端分离架构（SPA+RESTful/WebSocket）

由于本项目的要实现“一端搭建，多端共享”，未来会涉及到多端（手机、平板、电脑）观影，所以采用前后端分离的架构，充分利用前后端分离的优势。具体实现如下：

1.**前端独立应用：**

a.使用 Vue.js 构建单页面应用（SPA）

b.前端独立开发、构建和部署

c.通过 RESTful API 和 WebSocket 与后端通信

d.前端负责 UI 渲染、用户交互和客户端状态管理

2.**后端 API 服务：**

a.提供纯 API 接口，不涉及页面渲染

b.负责业务逻辑处理、实时通信和数据管理

c.采用无状态设计（除内存存储外），便于未来扩展

3.**通信协议：**

a.HTTP/REST 用于常规数据请求（房间管理、配置等）

b.WebSocket（通过 Socket.IO）用于实时同步和聊天

c.统一的数据交换格式（JSON）

**本项目采用前后端分离架构的优势**

- **高可维护性：** 前端和后端代码库完全解耦，可独立开发、测试和部署 。
- **更好的性能：**前端资源后续可独立部署在 CND 上，以减少服务器压力。
- **更好的用户体验：**SPA 提供更流畅的交互体验
- **多端支持：** 同一套 API 接口可以同时支持 Web、移动端和桌面客户端 。

### 2. 基于 DDD 思想的分层设计

水平上分为表现层、应用层、领域层、基础设施层。

**各层职责说明**

1.**表现层：**

a.用户界面渲染和交互处理

b.客户端状态管理

c.API 调用和 WebSocket 通信封装

d.本地数据验证和格式化

2.**应用层：**

a.请求路由和分发

b.身份验证和权限检查

c.业务逻辑编排

d.事务管理和一致性保证

e.WebSocket 连接管理和事件分发

3.**领域层：**

a.核心业务实体定义（Room, VideoState, Participant 等）

b.业务规则和领域逻辑

c.内存数据存储和管理

d.实体间的关系和聚合

**MVC 模式**

通过 MVC 模式，我们将表示层的交互（View/Controller）与领域层的业务逻辑（Model）分离，从而实现界面与业务的独立演化。

**前端 MVC 实现**

**Model：**Vuex 状态管理 + 组件响应式数据

- 存储应用状态（房间信息、用户信息、播放状态）
- 通过 actions/mutations 管理状态变更
- 与服务层交互获取和更新数据

**View：**Vue 模板和组件

- 渲染用户界面
- 接收用户输入事件
- 展示数据和状态变化

**Controller：**Vue 组件方法 + Vuex Actions

- 处理用户交互
- 调用服务层方法
- 更新 Model 状态

- 管理组件生命周期
- 后端 MVC 实现

**Model**：内存对象模型（Mock 模式）

- 业务实体类（Room, VideoState 等）
- 内存存储管理（模拟数据库操作）
- 为未来持久化存储提供兼容接口

**Controller**：路由控制器 + Socket.io 处理器

- 接收 HTTP 请求和 WebSocket 消息
- 参数验证和格式化
- 调用服务层处理业务逻辑
- 返回响应或广播事件

**Service**：业务逻辑服务层

- 实现具体的业务规则
- 协调多个领域对象协作
- 处理事务和一致性
- 为 Controller 提供简洁接口

### 3. 模块化单体架构与垂直分区

由于时间关系，本项目先从单体架构起步，尽快实现第一个版本。但未来有想法将视频转码（需要部署在高性能服务器上）、视频存储与分发（需要部署在网络高速，存储量大的服务器上）、后端（随意一台有公网的服务器即可）等功能提取出来作为微服务，所以即使采用单体架构，在架构上也要实现逻辑模块化，为未来微服务拆分做准备：

1.**模块划分原则**

**高内聚低耦合：**相关功能集中，模块间依赖最小化

**明确接口契约：**模块间通过定义良好的接口通信

**独立可测试：**每个模块可独立测试和验证

**渐进式拆分：**支持按需将模块拆分为独立微服务

2.**核心功能模块**

第一次迭代针对“参与社交影院”的基本流涉及到的功能，进行划分:

**房间管理模块 (Room Management)**

- 房间创建/解散

- 房间配置管理

**同步控制模块**

- 视频状态管理
- 同步事件分发
- 客户端状态对齐

**聊天互动模块**

- 消息收发

**3.模块通信机制**

- 同步调用：模块间方法调用（用于紧耦合操作）
- 事件驱动：通过事件总线进行解耦通信
- 共享存储：通过统一的内存存储管理器访问数据
- 接口隔离：每个模块对外提供明确的服务接口

**4. 数据管理策略选择——内存存储+Repository 抽象**

**选择理由：**

1.符合当前迭代目标：房间管理核心流程（创建/加入/退出/解散）优先跑通。

2.Repository 已提供“替换点”：RoomRepository 已封装存储细节；Service/Controller 基本不需要改动。

3.降低复杂度：避免过早引入 DB、迁移、连接池与运维。

**明确边界：哪些数据可以丢、哪些不能丢**

**可能需要持久化（中后期）：**

- 房间基本信息（name/capacity/password/announcement/status/creatorId/createTime）
- 参与者信息（participantId/nickname/role/状态）

**可以不持久化（实时场景可丢）：**

- 播放同步事件（play/pause/seek/heartbeat）
- 即时聊天消息（如果定位为“无历史记录聊天室”）

## 5.3. 系统逻辑结构

逻辑架构展示系统的软件模块组织结构，按架构层次（表示层、应用逻辑层、域层）划分，前后端分别描述。需要包图。

**1. 将系统拆分为子系统**

**1.1. 子系统拆分与核心定义**

按功能域拆分后的三个子系统，各自有清晰的职责边界，且通过标准化接口协作：

**聊天互动子系统：**

**核心职责：**负责所有消息的收发、广播、展示，保障实时性。

**包含的核心模块：**前端 ChatUI、后端 ChatController。

**核心数据模型：**消息内容、发送者、接收者、时间戳。

**同步播放子系统：**

**核心职责：**负责视频播放、暂停、进度同步、视频源管理，依赖 artPlayer 第三方组件。

**包含的核心模块：**前端 VideoPlayerUI/VideoSourceSettingUI、后端 VideoStateController/SyncController。

**核心数据模型：**VideoState（URL、进度、状态）、SyncEvent。

**房间管理子系统：**

**核心职责：**负责房间的创建、加入、解散、成员管理，是聊天 / 视频子系统的基础载体。

**包含的核心模块：**前端 RoomCreationUI/RoomJoiningUI/RoomDismissalUI、后端 RoomDismissalController、Room/Participant 实体。

**核心数据模型：**Room（ID、成员、配置）、Participant（用户信息）。

### 1.2. 分层设计

分层是从 “前端→后端→基础设施” 的纵向维度拆分，保证各层职责单一、依赖清晰，所有子系统（聊天互动 / 同步播放 / 房间管理）均遵循该分层规则：

**表现层（UI 层）：**

**核心职责：**用户交互、界面渲染、操作触发。

**所有子系统的共性实现：**聊天 UI（消息输入 / 展示）、视频 UI（播放器 / 画质设置）、房间 UI（创建 / 加入 / 成员列表）

**应用层（业务层）：**

**核心职责：**子系统核心业务逻辑、流程编排、权限校验。

**所有子系统的共性实现：**聊天服务（消息收发 / 广播）、视频服务（推流 / 同步）、房间服务（生命周期管理）。

**领域层（核心层）：**

**核心职责：**子系统核心实体、领域规则、数据模型。

**所有子系统的共性实现：**聊天领域（Message 实体）、视频领域（VideoStream 实体）、房间领域（Room/Participant 实体）。

**数据层（核心层）：**

**核心职责：**技术能力支撑（存储、通信、第三方依赖）。

**所有子系统的共性实现：**数据库（房间 / 消息存储）、WebSocket/HTTP（通信）、artPlayer（视 频播放）、CDN（视频资源）。

### 1.3. 分区设计

分区是从 “业务功能” 的横向维度拆分，将系统划分为 3 个独立业务分区（子系统），每个分区内包含完整的分层逻辑，且分区间通过标准化接口交互：

**房间管理分区：**

**核心边界：**仅负责房间生命周期（创建 / 解散）、成员管理（加入 / 退出）、房间状态维护。

**专属分层组件：**表现层：RoomUI；应用层：RoomService；领域层：Room/Participant；基础设施 层：房间数据库。

**对外交互接口：**提供 “房间状态查询 / 变更” 接口，供聊天 / 视频分区调用。

**同步互动分区：**

**核心边界：**仅负责消息收发、广播、历史查询、消息格式校验。

**专属分层组件：**表现层：ChatUI；应用层：ChatService；领域层：Message；基础设施层：消息 数据库 + WebSocket。

**对外交互接口：**依赖房间分区的 “成员列表” 接口，仅向房间内成员广播消息。

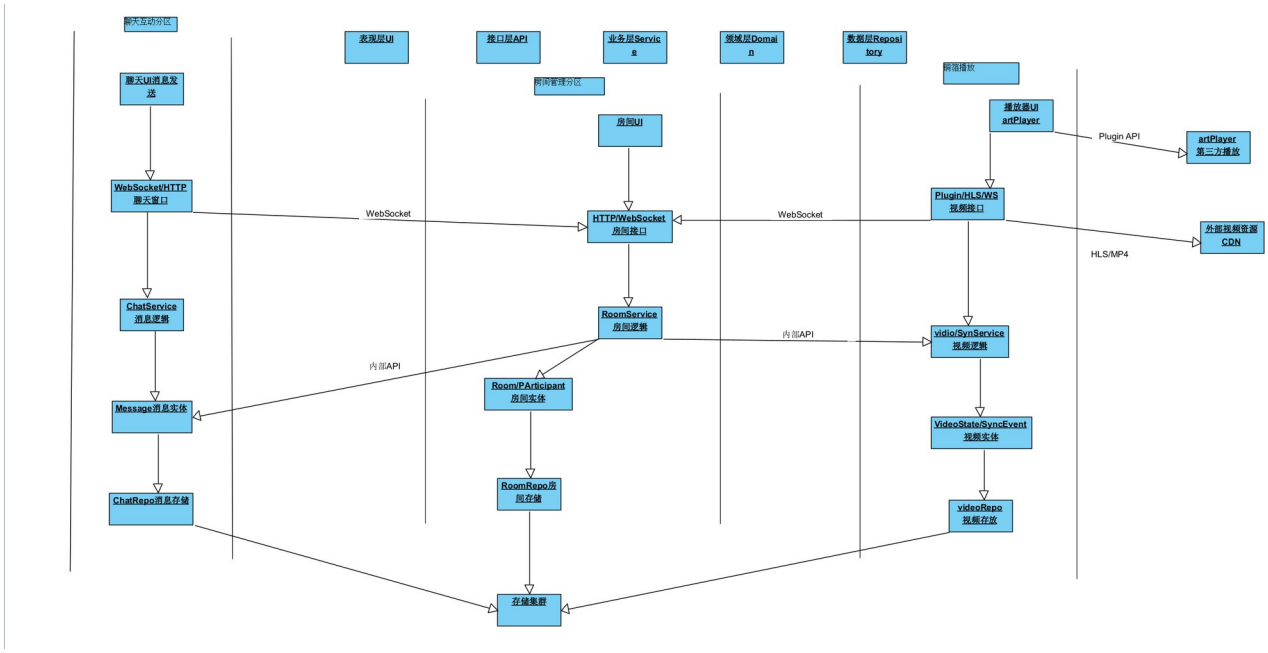
**聊天管理分区：**

**核心边界：**仅负责视频播放 / 暂停、推流 / 拉流、播放同步、视频参数配置。

**专属分层组件：**表现层：VideoUI；应用层：VideoService；领域层：VideoStream；基础设施层： artPlayer+CDN。

**对外交互接口：**依赖房间分区的 “房间状态” 接口，仅在有效房间内提供视频服务。

### 1.4. 组合分层和分区




a.纵向是分层（表现层→数据层）。

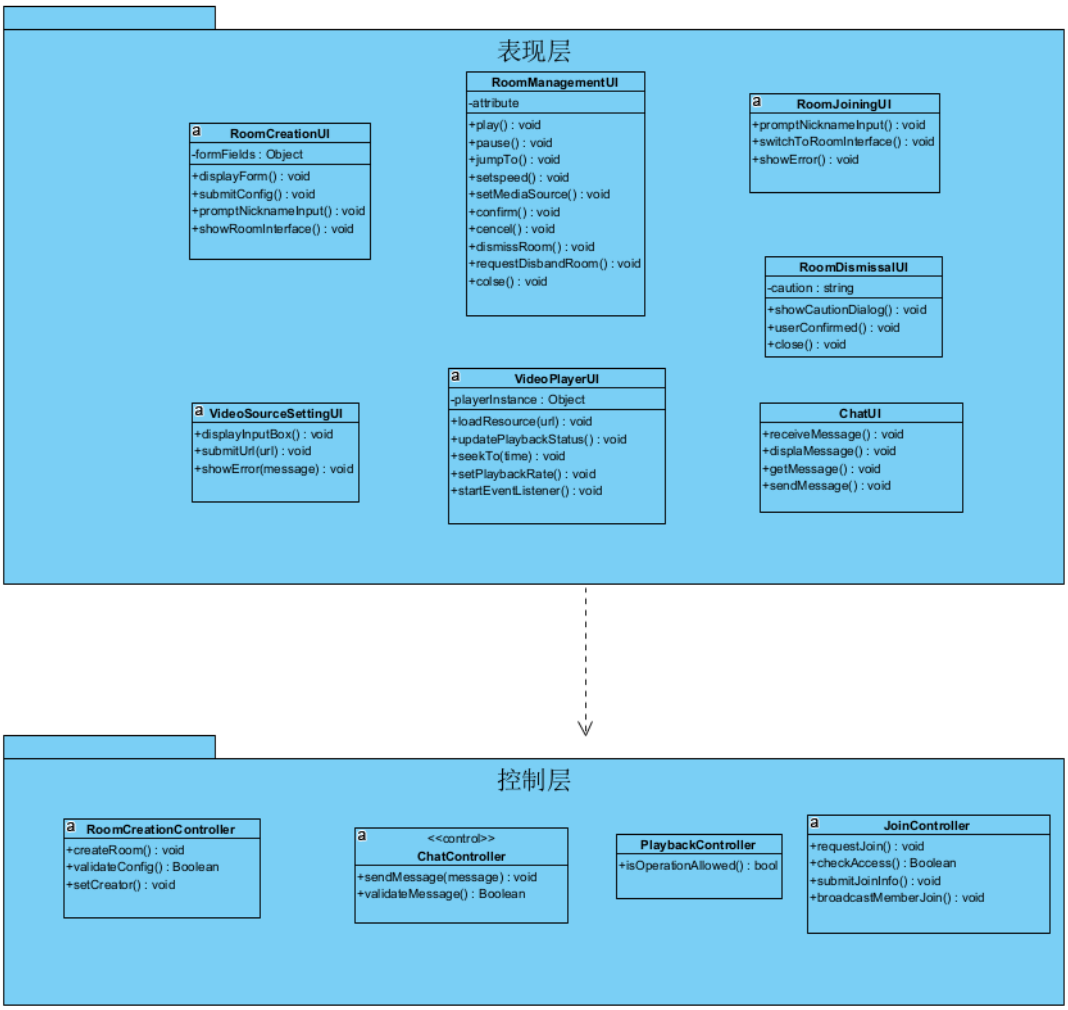
b.横向是分区（房间 / 聊天 / 视频）。

c.每个分区内的组件严格按分层调用。

d.分区间仅通过业务层 / 接口层交互，无跨层依赖。

e.第三方依赖仅关联对应分区的指定层级，边界清晰。

## 2. 前端包结构




### 2.1. 表现层

这一层是用户直接操作的界面组件，只处理“界面渲染、用户输入 / 反馈”，不包含业务逻辑，模块包括：

- **RoomCreationUI**：房间创建的界面，负责显示创建表单、提示用户输入房间名、展示房间界面等；
- **RoomManagementUI**：房间管理的界面，处理播放、暂停、请求解散房间等操作的界面交互；
- **RoomJoiningUI**：房间加入的界面，负责提示用户输入房间号、切换房间界面等；
- **RoomDismissalUI**：房间解散的界面，展示提示信息、确认弹窗等；
- **VideoSourceSettingUI**：视频源设置的界面，处理输入框展示、提交 URL 等；
- **VideoPlayerUI**：视频播放器的界面，负责播放器初始化、状态更新等；

·**ChatUI**：聊天界面，处理消息收发、展示等。

### 2.2. 控制层

这一层是 “表现层” 和 “后端 / 数据层” 的中间件，接收 UI 层的请求、处理业务逻 辑（验证、调用接口等），模块包括：

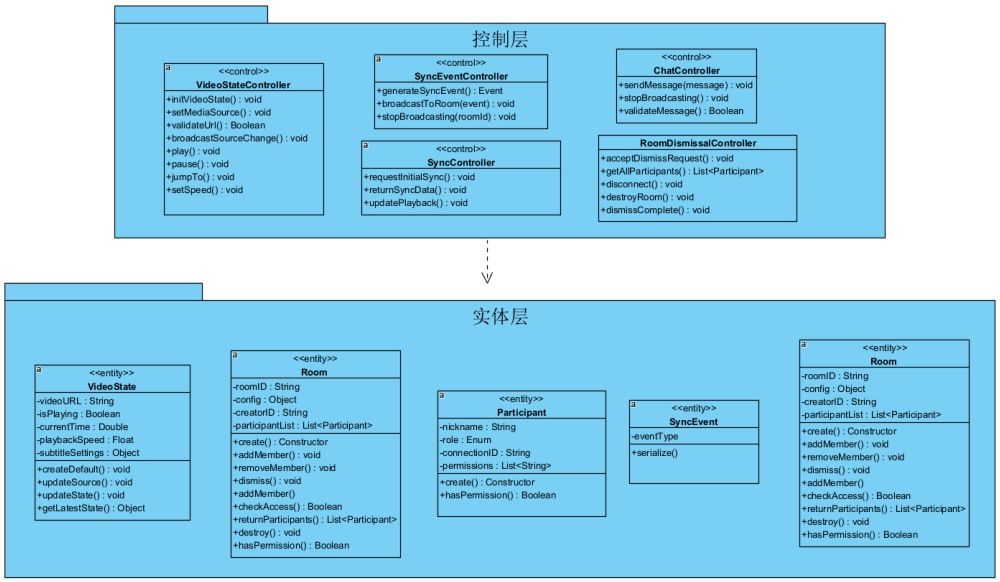
- **RoomCreationController**：处理房间创建的逻辑（创建请求、验证表单、设置创建者等）；
- **ChatController**：处理聊天的逻辑（发送消息、验证消息合法性等）；
- **PlaybackController**：处理播放控制的逻辑（判断操作是否允许等）；
- **JoinController**：处理房间加入的逻辑（请求加入、验证权限、广播成员加入等）。

### 2.3. 架构特点

这个设计是前端分层解耦的典型方式：

- 表现层只做 “界面交互”，不碰业务逻辑；
- 控制层只做 “逻辑处理”，不碰界面渲染；
- 两者通过 “请求 - 响应” 的方式协作（比如 RoomCreationUI 的操作，会调用 RoomCreationController 的逻辑）。

## 3. 后端包结构




### 3.1. 控制层

这一层是后端的 “逻辑调度中心”，接收前端请求、处理业务规则（比如权限校验、事件广播），并操作实体层的数据，模块包括：

- **VideoStateController**：视频状态控制，负责初始化视频状态、设置播放源、控制播放 / 暂停 /

跳转等逻辑；

- **SyncController**（两个）：同步控制，负责生成同步事件、广播同步数据、更新播放状态等，保障多用户的播放同步；
- **ChatController**：聊天控制，处理消息发送、广播、合法性验证；
- **RoomDismissalController**：房间解散控制，处理解散请求、成员断开连接、销毁房间等逻辑。

### 3.2. 实体层

这一层是后端的 “数据载体”，定义了系统中核心对象的结构和基础操作，模块包括：

- **VideoState**：视频状态模型，存储视频 URL、播放状态、进度、倍速等核心数据；
- **Room**：房间模型，存储房间 ID、配置、创建者、成员列表等房间信息，提供成员管理、解散等基础方法；
- **Participant**：参与者模型，存储用户昵称、角色、权限等信息；
- **SyncEvent**：同步事件模型，定义同步事件的类型，用于多用户播放同步的消息载体。

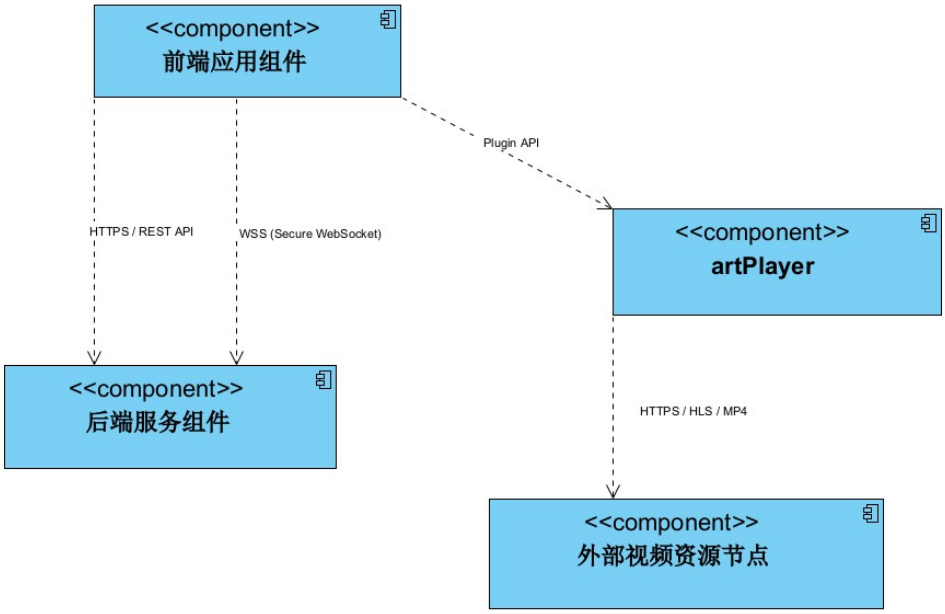
### 3.3. 架构特点

这个设计是后端“逻辑 - 数据分离”的典型方式：

- 控制层只做“业务逻辑”，不直接存储数据，而是调用实体层的方法操作数据；
- 实体层只做“数据存储 + 基础操作”，不包含复杂业务逻辑；
- 两者协作：控制层处理业务规则后，通过实体层的方法读写数据（比如 RoomDismissalController 调用 Room 的 dismiss()方法完成房间解散）。

## 5.4. 系统组件结构

产出组件图、并定义各组件之间的通信方式，如：标出这两个组件之间是通过 HTTP/JSON 还是 WebSocket 接口连接的。识别第三方依赖，如果使用了第三方的视频播放（已知视频播放用的 artPlayer 第三方播放器）插件或存储服务，它们在组件图中会被表现为外部组件。




1. 核心组件说明

- **前端应用组件：**系统的前端交互界面（对应之前的 “表现层”），是自研业务组件。
- **后端服务组件：**系统的后端逻辑处理（对应之前的 “控制层 + 实体层”），是自研服务组件。
- **artPlayer：**第三方视频播放插件（负责视频渲染、播放控制），是外部依赖组件。
- **外部视频资源节点：**提供视频源的外部服务（如视频存储 / CDN），是外部资源组件

2. 组件间通信方式

- **前端应用组件 <--> 后端服务组件：**通信方式：HTTPS/REST API（同步请求，如房间创建、加入） + WS (Secure WebSocket)（异步通信，如聊天、播放同步）；
- **前端应 <--> artPlayer：通信方式：**Plugin API（artPlayer 提供的插件接口，用于前端控制播放器）；
- **artPlayer <--> 外部视频资源节点：**通信方式：HTTPS / HLS / MP4（拉取视频资源，支持 HLS 流、MP4 文件）。

3. 第三方依赖识别

图中明确的第三方依赖是 artPlayer（视频播放插件），属于 “外部组件”；此外 “外部视频资源节点” 也属于外部依赖（非自研）。

## 5.5. 物理部署结构

本节描述“线上观影厅”系统在物理节点上的分布、运行环境配置以及网络通信协议。鉴于本次迭代聚焦于核心的“参与社交观影”用例，且不涉及服务器端的视频转码与存储，

系统采用 集中式单体部署（Centralized Monolithic Deployment） 策略。

架构的核心设计原则是控制流与数据流分离：应用服务器仅负责轻量级的 WebSocket 信令交换（控制流）和前端资源分发，而高带宽消耗的视频数据（数据流）则由客户端直接通过直链向第三方源请求，从而在低成本硬件上实现高并发的同步观影体验。

### 1. 部署拓扑概述

系统部署拓扑主要包含三个逻辑节点：客户端节点、应用服务器节点以及第三方资源节点。

1.**客户端节点：**

a.承载用户交互界面与本地播放逻辑。

b.不依赖任何插件安装，运行于标准 Web 浏览器环境中。

c.直接负责视频流的拉取与解码渲染。

2.**应用服务器节点：**

a.采用单机部署，运行于 Linux 环境。

b.作为系统的唯一入口，集成 Nginx 反向代理与 Node.js 应用服务。

c.负责托管前端静态资源（SPA）及处理 WebSocket 实时信令，所有业务状态（房间、成员、播放进度）均暂存于服务器进程内存中。

3.**第三方资源节点：**

a.指代外部的视频托管服务器或 CDN。

b.在本次迭代中，系统不自行存储视频文件，而是由客户端根据 URL 直接访问该节点获取 .mp4 或 .m3u8 媒体流。

### 2. 节点运行环境说明

1.**客户端节点**

**硬件要求：**具备基础音视频解码能力的 PC、笔记本或移动设备。

**软件环境：**

- 支持 ES6+ 及 WebSocket 标准的现代浏览器。
- 兼容列表：Chrome 90+, Firefox 88+, Safari 14+, Edge 90+。

**部署制品：**

- Vue.js 单页应用（SPA）构建后的静态文件包（HTML/CSS/JS）。
- 集成 HTML5 播放器组件（Video.js/ArtPlayer）。

2. **应用服务器节点**

**操作系统：**Windows (演示与开发时用 Windows 以保持与开发环境一致)。

**Web 服务器（反向代理）：**

- **组件：**Nginx 。
- **职责：**

  **SSL 卸载**（SSL Termination）：处理 HTTPS/WSS 加密解密，保障通信安全。

  **静态资源服务：**直接响应前端 SPA 的静态文件请求，降低 Node.js 负载。

  **反向代理：**将 API 请求和 WebSocket 连接转发至后端的 Node.js 端口。

**应用运行时：**

- **组件：**Node.js Runtime (v25.2.1)。
- **进程管理：**使用 PM2 或 Systemd 进行进程守护，确保服务崩溃后自动重启。
- **部署制品：**后端 Express + Socket.IO 单体应用包。

**数据存储：**

- **组件：**In-Memory Store (Node.js 堆内存)。
- **说明：**利用 JS 对象或 Map 结构存储 Room 和 VideoState 实例，重启即销毁。

3. 网络通信协议与端口规划

系统严格遵循现代浏览器的安全策略，生产环境强制使用加密传输协议。

| 通信链路 | 协议 | 端口 (对外/对内) | 说明 |
|---|---|---|---|
| Client ↔ Nginx | HTTPS (HTTP/1.1 or 2) | 443 / 80 | 前端页面加载、REST API 调用（如创建房间、鉴权）。80 端口强制重定向至 443。 |
| Client ↔ Nginx | WSS (Secure WebSocket) | 443 | Socket.IO 实时双向通信（同步播放进度、聊天）。复用 443 端口。 |
| Nginx ↔ Node.js | HTTP / WS | 3000 (Localhost) | 内部纯文本通信。Nginx 将外部流量转发至本地 3000 端口。 |
| Client ↔ External Video | HTTPS / HTTP | 443 / 80 | **视频流直连**。客户端直接请求外部 URL，流量不经过应用服务器。 |

4. 部署与运维约束

1.**安全性约束：**

由于涉及音视频同步与未来可能的媒体权限，部署域名必须配置有效的 SSL 证书（如 Let's Encrypt），否则浏览器将拦截 WebSocket 连接或混合内容（Mixed Content）。

2.**状态易失性约束：**

鉴于当前架构无持久化数据库，服务器应用更新或重启时，所有活跃房间和用户连接将断开， 数据将重置。运维操作（如发布新版本）需安排在低峰期进行。

3.**并发限制：**

受限于单 Node.js 进程的内存上限（通常约 1.5GB）及单机 CPU 性能，单台服务器支持的同 时在线人数存在物理瓶颈（预计支持百级并发连接），需通过负载测试确定具体阈值。

## 5.6. 并发与分布处理

1. 识别内部并发性

- **同步播放子系统：**需要持续处理 PLAY/PAUSE/SEEK 等同步事件，严格要求低延迟（<100ms），必须独立并发处理，以确保所有观众播放状态严格一致
- **聊天互动子系统：**消息收发完全独立于视频状态，可异步处理而不影响主流程，符合内在并发特征，可分配独立处理路径
- **房间管理子系统：**观众加入/离开是随机事件，需要独立处理连接生命周期，不能阻塞同步事件处理；创建/解散是低频操作，虽涉及多对象协作但可通过单线程顺序处理，只需保证事务性而非真正的并发

2. 定义并发任务

2.1. 并发任务划分

**（1）同步播放子系统：**

**核心职责：**负责视频播放状态同步、进度同步、视频源管理，确保所有观众体验一致

**关键控制类：**

- SyncEventController：生成同步事件(Sync_Event)并向房间内所有观众广播事件
- VideoStateController：维护 VideoState 作为唯一真实数据源，更新播放状态
- PlaybackController：验证控制操作权限及参数有效性

**状态机对应：**播放中→暂停中→同步控制循环

**关键特征：**高优先级，低延迟要求（<100ms），需严格保证事件顺序

**责任来源：**"系统生成 Sync_Event(PLAY)并广播"；"系统更新 VideoState，作为新的'唯

一真实数据源'"

**（2）聊天互动子系统：**

**核心职责：**负责所有消息的收发、广播、展示，保障社交互动实时性

**关键控制类：**

・ChatController：验证消息内容合规性，将消息广播给在线成员

**状态机对应：**并行子状态 2：聊天互动

**关键特征：**独立于播放状态，可批量处理，容忍短暂延迟（<1s）

**责任来源：**"系统验证消息内容合规"；"系统将消息广播给房间内在线成员"

**（3）房间管理子系统：**

**核心职责：**负责房间的创建、加入、解散、成员管理，是其他子系统的基础载体

**关键控制类：**

・JoinController：建立实时连接并加入在线列表，广播"观众加入"通知

・RoomCreationController：校验创建参数合法性，创建观影室并生成唯一标识

・RoomDismissalController：强制断开所有连接，销毁房间实例及其 VideoState

・SyncController：读取 VideoState 并向新加入观众下发初始化数据

**状态机对应：**房间生命周期（创建→活跃→解散）

**关键特征：**需要事务性保证，特别是房间解散操作，中等延迟容忍度（<500ms）

**责任来源：**"强制断开所有成员连接，停止广播"；"系统向成员广播'观众加入'通知"

### 2.2. 控制线程处理并发

本系统采用事件驱动架构，在 Node.js 单线程模型基础上，通过精心设计的事件队列与优先级调度机制，实现了多条逻辑控制线程的并行处理能力。以下详细说明系统中的关键控制线程：

**（1）同步播放控制流**

当用户触发 PLAY 操作时，控制线程遵循以下路径：

・起始于 PlaybackController，负责验证控制操作权限及参数有效性

・传递至 VideoStateController，负责更新播放状态，维护 VideoState 作为唯一真实数据源

・终止于 SyncEventController，负责生成同步事件(Sync_Event)并向房间内所有观众广播

此控制流具有高优先级特性，严格要求端到端处理延迟低于100毫秒，并需保证所有同步事件的严格顺序性，以确保所有观众的播放状态保持一致。

**（2）观众加入控制流**

当观众加入房间时，控制线程遵循以下路径：

- 起始于JoinController，负责建立实时连接并加入在线列表
- 经过连接建立与成员注册过程
- 终止于SyncController，负责读取当前VideoState并向新加入观众下发初始化数据，确保新成员 获得正确的播放状态

此控制流需要事务性保证，特别是连接建立与状态同步必须作为一个完整单元处理，延迟容忍度为500毫秒以内。该流程不能阻塞同步播放控制流的执行，体现了子系统间的独立处理能力。

**（3）聊天消息控制流**

当用户发送聊天消息时，控制线程遵循以下路径：

- 起始于ChatController，负责验证消息内容合规性
- 终止于消息广播环节，将验证通过的消息广播给房间内在线成员

此控制流完全独立于视频状态，可异步处理而不影响主流程，符合内在并发特征，延迟容忍度为1秒以内。消息处理与播放同步操作互不干扰，确保社交互动不影响核心观影体验。

## 5.7. 处理器分配

### 1. 系统部署架构概述

**核心设计理念：**显式进程分配替代隐式事件循环调度

- 专用CPU核心绑定确保关键任务执行确定性
- 通过共享存储（Redis）解决进程间数据一致性问题

**物理部署架构：**主进程（Master）：负责进程管理与健康监控

- 业务进程组：按子系统功能分离，每组专用CPU核心
- 共享状态层：Redis集群替代内存存储，实现跨进程状态同步
- 负载均衡层：Nginx实现基于房间ID的亲和性路由

### 2. 子系统的处理需求

**同步控制进程：**

- CPU需求：高，每秒处理10+同步事件/房间
- 内存需求：中，每个房间状态对象<1KB

- 实时性要求：严格<100ms 端到端延迟
- 容错要求：高，状态不一致导致用户体验崩溃

**房间管理进程：**

- CPU 需求：低，主要开销在 I/O 等待
- 内存需求：高，缓存房间元数据
- 事务要求：强一致性，房间状态变更必须原子
- 吞吐量要求：每秒 50+房间操作请求

**聊天服务进程：**

- CPU 需求：中高，内容过滤为 CPU 密集型
- 带宽需求：高，消息广播产生大量网络 I/O
- 一致性要求：最终一致性，允许短暂延迟
- 伸缩性要求：支持动态扩展应对突发流量

3. 分配策略

**物理进程分配策略：**

**主进程 (Master)：**单实例，仅负责进程生命周期管理

**同步控制进程组：**2 个专用进程，独占 CPU 核心 0 和 1 每进程负责约 400 个活跃房间

- 严格禁止执行 I/O 阻塞操作
- 优先级设置为实时级别（nice -20）

**房间管理进程组：**2 个专用进程，独占 CPU 核心 2 和 3 通过 Redis 共享房间元数据

- 支持动态负载均衡

**聊天服务进程：**1 个专用进程，共享 CPU 核心 4-7

- 支持水平扩展（根据负载自动增加实例）
- 具备背压控制和优雅降级能力
- 关键消息优先处理

4. 通信基础设施

**状态共享机制：**

- 使用 Redis Hash 存储 Room 对象
- 采用 Redis 事务保证数据变更原子性

- 本地缓存策略：LRU 缓存+TTL(100ms)+写失效
- 版本控制：每次状态变更增加版本号，解决冲突

**事件同步机制：**

- Redis Pub/Sub 实现跨进程事件广播
- 专用频道命名：room:{roomId}:events
- 消息格式：Protocol Buffers 序列化
- 背压控制：消费者积压>1000 条时暂停生产

# 第6章详细设计

## 6.1. 引言

本章的目的是将上一阶段的架构设计与分析模型（BCE/MVC/DDD 分层、模块化单体、核心用例 UC-01）进一步细化为可直接编码的蓝图。本章不仅描述业务概念类（如 Room、Participant），还必须给出实现细节：

- 核心类的属性/方法签名、可见性、初始值与关键约束。
- 为了落地实现必须引入的“解决方案域类”（如 Socket 处理器、房间连接管理器、事件工厂、时间同步工具）。
- 动态行为的精确定义：关键用例的序列图（箭头上的文字必须是具体方法名）与核心对象状态机。
- 内部接口定义：HTTP API + Socket Events 的协议契约与错误处理。

## 6.2. 类设计与精化

本节按架构设计中定义的三个垂直业务子系统进行详细类设计，严格遵循"单一职责"和"高内聚低耦合"原则：

**房间管理子系统**

**核心职责：**房间生命周期管理、成员身份认证与权限控制

**同步播放子系统**

**核心职责：**视频状态管理、多端播放同步、时间轴对齐

**聊天互动子系统**

**核心职责：**实时消息广播、内容安全校验

### 1. 房间管理子系统

**领域模型**

**Room**

**关键属性**

- id: String（6 位房间号）
- name: String
- capacity: number
- password: String | null
- announcement: String
- status: 'waiting' | 'playing' | 'closed'

- videoState: VideoState
- participants: Map<String, Participant>
- creatorId: String
- createTime: Date
- updateTime: Date

关键方法

- hasPassword(): boolean
- validatePassword(password: string): boolean
- isFull(): boolean
- getOnlineCount(): number
- addParticipant(participantData: object): Participant
- removeParticipant(participantId: string): boolean
- getParticipant(participantId: string): Participant | undefined
- getCreator(): Participant
- visCreator(participantId: string): boolean
- updateConfig(config: object): void
- setStatus(status: string): void
- close(): void
- toListJSON(): object
- toDetailJSON(): object

**Participant**

关键属性

- id: String
- nickname: String
- role: 'creator' | 'viewer'
- status: 'online' | 'offline'
- joinTime: Date
- socketId: String | null

关键方法

- isCreator(): boolean
- setStatus(status: string): void
- setSocketId(socketId: string): void
- toJSON(): object

应用服务

- RoomService：参数验证、业务流程编排、权限校验（创建者权限）。
- RoomController：HTTP 请求处理与统一响应。

基础设施

- RoomRepository：内存仓库 Map<String, Room>，提供 create/findById/findAll/update/delete/exists/search/count。

2. 同步播放子系统

领域模型

**VideoState**

关键属性

- source: String | null
- status: 'playing' | 'paused' | 'stopped'
- progress: number
- playbackRate: number
- subtitle: String | null
- lastUpdateTime: number（时间戳）

关键方法

- setSource(source: string | null): void
- setStatus(status: string): void
- setProgress(progress: number): void
- setPlaybackRate(rate: number): void
- getCurrentProgress(): number
- toJSON(): object

**SyncEvent（值对象 / DTO）**

属性：

・type: SyncEventType

・roomId: string

・operatorId: string（发起人 participantId）

・serverTime: number（毫秒时间戳，服务端生成）

・payload: object（不同事件类型内容不同）

方法：

・toJSON(): object

补充：事件类型枚举 **SyncEventType**

・CHANGE_SOURCE

・PLAY

・PAUSE

・SEEK

・CHANGE_RATE

・CHANGE_SUBTITLE

应用服务

**SyncService（业务服务）**

职责

・校验权限：只有房主/控制者可执行同步控制（第一次迭代默认房主）。

・校验参数合法性。

・更新 Room.videoState。

・生成 SyncEvent 并交给广播层发送。

关键属性

・roomService: RoomService（复用现有模块能力）

关键方法

・changeSource(roomId: string, operatorId: string, sourceUrl: string | null): Promise<{ room: object, event: object }>

・play(roomId: string, operatorId: string): Promise<{ room: object, event: object }>

・pause(roomId: string, operatorId: string): Promise<{ room: object, event: object }>

・seek(roomId: string, operatorId: string, progress: number): Promise<{ room: object, event: object }>

・changeRate(roomId: string, operatorId: string, rate: number): Promise<{ room: object, event: object }>

・changeSubtitle(roomId: string, operatorId: string, subtitle: string | null): Promise<{ room: object, event: object }>

・getInitState(roomId: string): Promise<{ videoState: object, serverTime: number }>（给新加入者下发初始化数据）

返回值中携带 event 是为了让上层（socket handler）统一广播。

**基础设施**

**SyncGateway / SyncSocketHandler（Socket 事件处理器）**

**职责**

・监听 Socket.IO 事件（例如 sync:play、sync:pause）。

・将事件参数转给 SyncService。

・将 SyncEvent 广播到房间频道 room:{roomId}。

**关键属性**

・io: Server（Socket.IO server 实例）

・syncService: SyncService

**关键方法**

・register(io: Server): void

・handlePlay(socket, data, ack): void

・handlePause(socket, data, ack): void

・handleSeek(socket, data, ack): void

・handleChangeRate(socket, data, ack): void

・handleChangeSubtitle(socket, data, ack): void

・handleChangeSource(socket, data, ack): void

・handleInitRequest(socket, data, ack): void（新成员初始化同步）

**TimeSyncUtil（工具类，解决方案域）**

**职责**

・提供服务端时间戳生成与时间差计算（用于“基准时间戳”策略）。

方法

・nowMs(): number

・calcExpectedProgress(baseProgress: number, baseTimeMs: number, nowMs: number, playbackRate: number): number

设计模式应用：

・Observer/Publish-Subscribe：socket.io room 广播实现发布订阅模型

・Factory：建议实现 SyncEventFactory 创建不同类型事件对象

3. 聊天互动子系统

领域模型

**ChatMessage（值对象 / DTO）**

属性

・messageId: string

・roomId: string

・from: { participantId: string, nickname: string }

・content: string

・sentAt: string（ISO8601）

方法

・toJSON(): object

应用服务

**ChatService（业务服务）**

职责

・参数校验（roomId、participantId、nickname、content、频率）。

・房间与成员关系校验（强绑定）：room 是否存在/是否关闭

・participant 是否属于该 room

・生成消息 DTO（带 messageId、sentAt）。

关键属性

・roomService: RoomService

关键方法

・joinRoomChat(roomId: string, participantId: string, nickname: string): Promise<{ channel: string }>

- leaveRoomChat(roomId: string, participantId: string): Promise<boolean>
- sendMessage(roomId: string, participantId: string, content: string): Promise<{ message: object }>

基础设施

**ChatGateway（Socket 事件处理器）**

职责

- namespace：/chat
- 事件：room:join、room:leave、message:send
- 通过 socket.join(room:{roomId}) 将连接加入频道
- io.to(room:{roomId}).emit('message:new', dto) 广播消息

关键属性

- io: Server
- chatService: ChatService

关键方法

- register(io: Server): void
- handleJoin(socket, data, ack): void
- handleLeave(socket, data, ack): void
- handleSend(socket, data, ack): void

4. 引入解决方案

分析阶段类图通常只包含业务概念（Room、User），但落地实现必须引入以下“工程类”。

**Socket 处理器（Gateways）**

- SyncGateway（处理同步控制事件、广播 Sync_Event）
- ChatGateway（处理聊天事件、广播消息）

它们与 HTTP Controller 并列，属于后端 MVC 中的 Controller/Handler。

5. 应用设计模式

**Singleton（单例模式）**

- RoomRepository 单例: 内存中只有一个数据存储，所有对房间的访问操作都基于同一份数据
- RoomService 单例: 业务逻辑状态一致，避免多个 Service 实例各自维护不同的房间

状态

- SyncService 单例: 同步控制需要统一的权限检查、状态管理
- ChatService 单例: 需要统一管理消息队列、频率限制记录

**Observer / Publish-Subscribe（观察者/发布订阅）**

- socket.io room 广播天然是发布订阅模型：发布者：服务端（SyncGateway/ChatGateway）

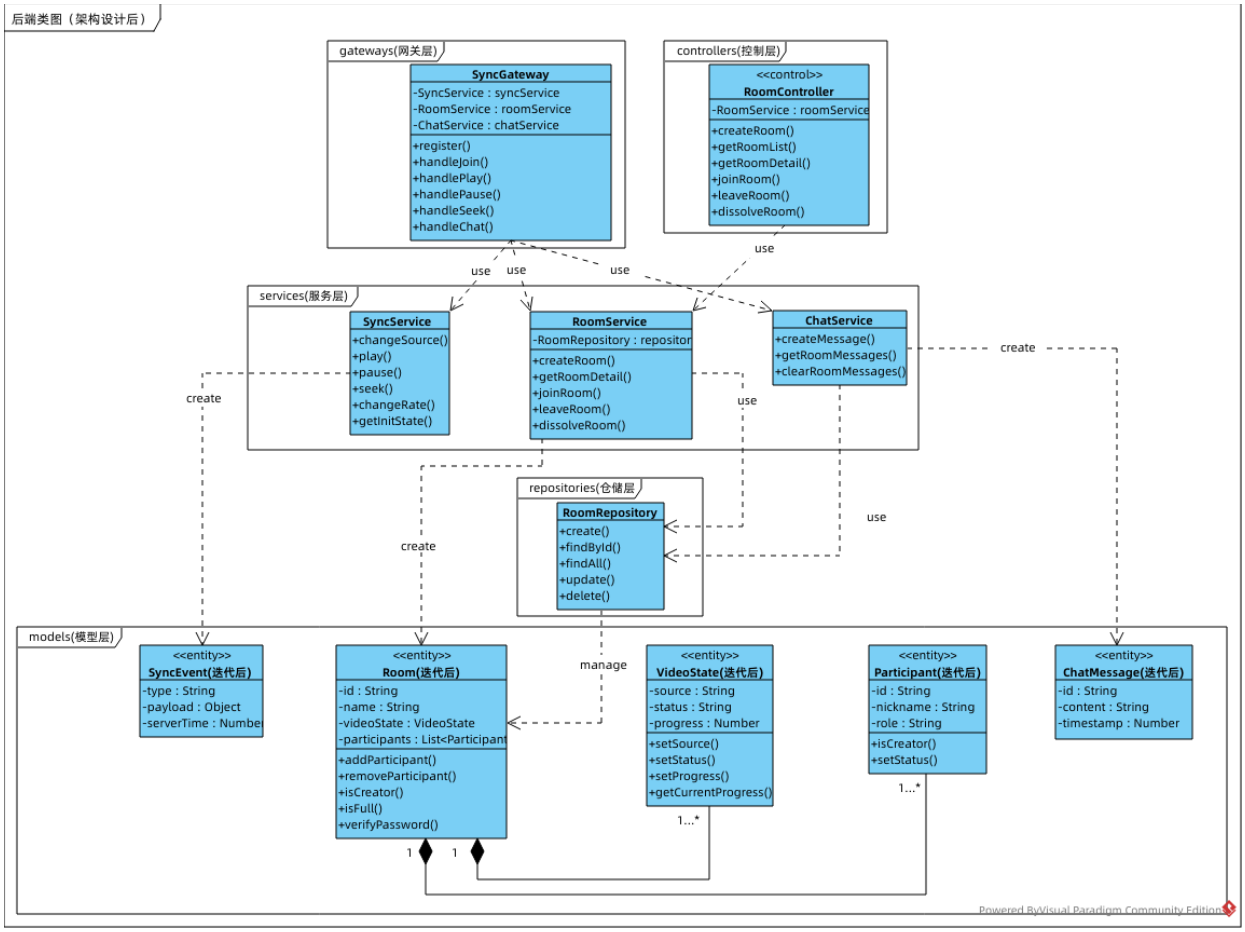
  订阅者：房间内各客户端

**Repository（仓储模式）**

- 封装数据访问逻辑，第一次迭代提供 RoomRepository。
- 同步/聊天第一次迭代不需要额外持久化仓库，但未来可扩展：SyncRepository（记录事件/回放）

  ChatRepository（消息历史）

**最终后端类图：**

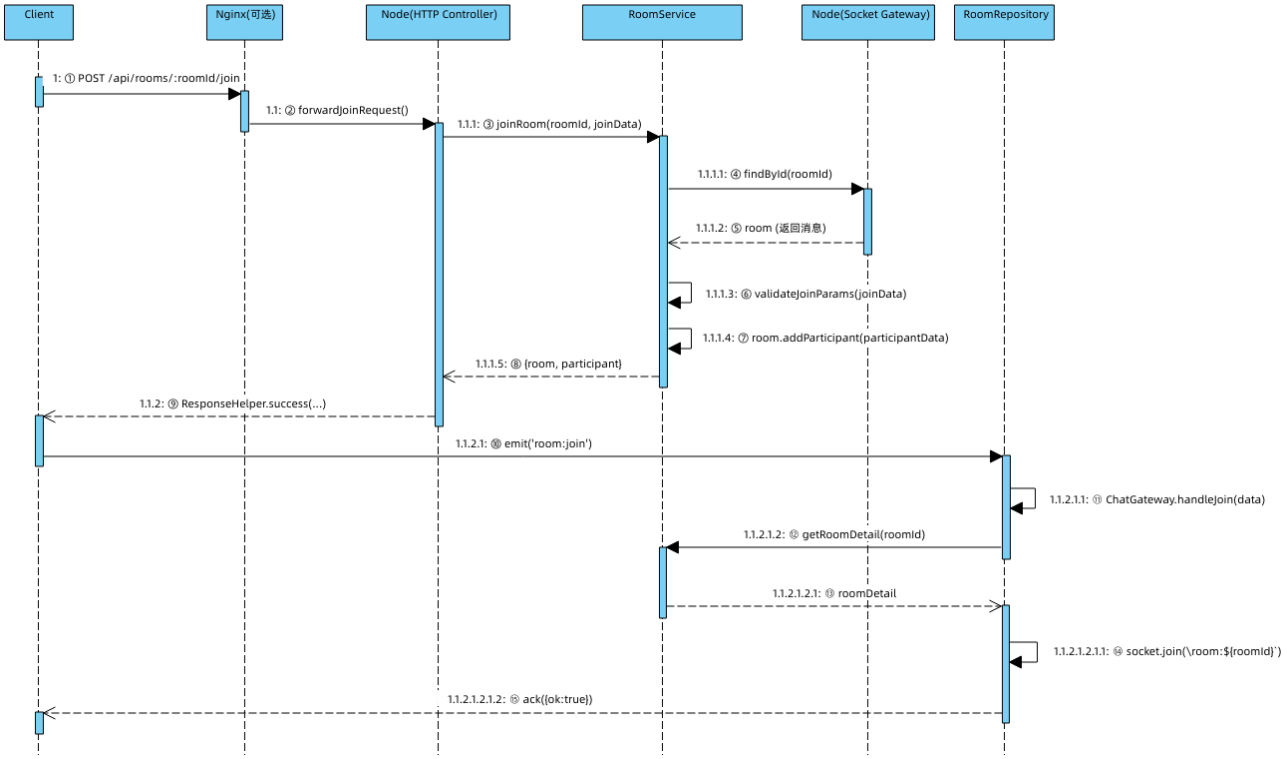


## 6.3. 动态模型设计

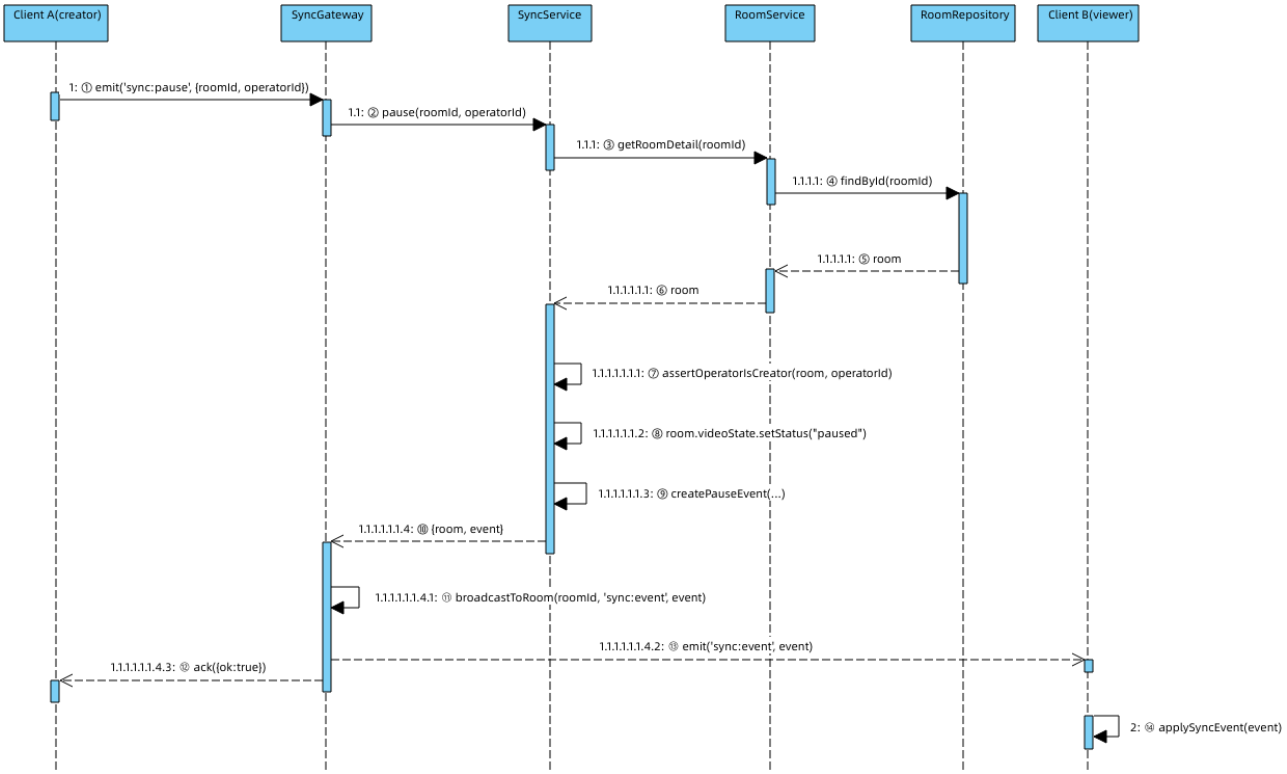
本节描述经过架构设计后的对象运行时交互与状态变化

### 1. 关键用例的序列图

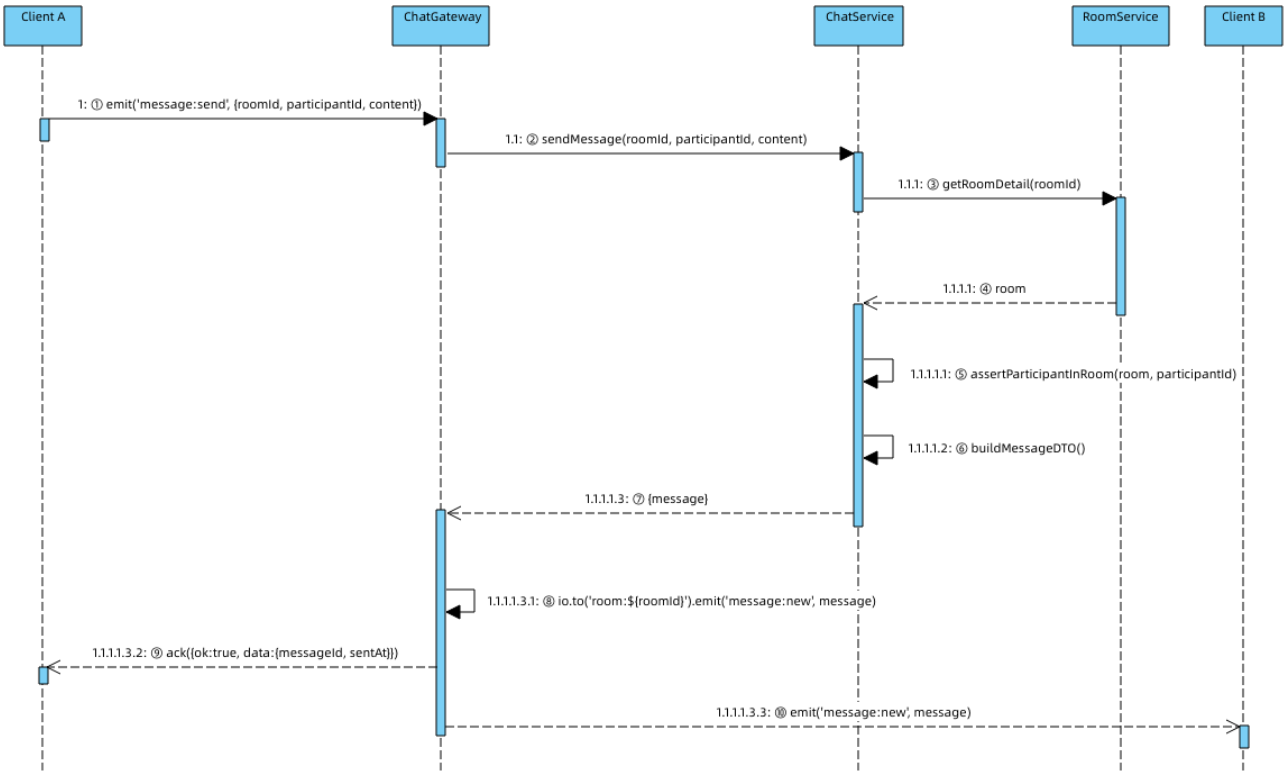
加入房间（HTTP/Socket）




同步播放




聊天互动




## 6.4. 接口与人机交互设计

本节定义前后端通信协议。

### 1. 内部接口定义

1.1. HTTP API

创建房间

创建一个新的线上观影室。

**请求路径：**/api/rooms

**请求头：**

| Header | Value | 必填 | 说明 |
|---|---|---|---|
| Content-Type | application/json | 是 | 请求体格式为 JSON |

**请求体参数：**

| 参数 | 类型 | 必填 | 说明 |
|---|---|---|---|
| name | string | 是 | 房间名称，长度限制 1-50 字符 |
| capacity | number | 否 | 房间人数上限，范围 2-100，默认值 10 |
| password | string | 否 | 房间密码，最长 20 字符，不传则无密码 |
| announcement | string | 否 | 房间公告，最长 500 字符 |
| creatorNickname | string | 是 | 房间创建者昵称 |

**成功响应：**


```
{
  "success": true,
  "code": 201,
  "message": "房间创建成功",
  "data": {
    "room": {
      "id": "123456",
      "name": "周末观影派对",
      "capacity": 20,
      "currentCount": 1,
      "hasPassword": true,
      "announcement": "欢迎来到我的观影室！",
```

```
    "status": "waiting",
    "videoState": {
      "source": null,
      "status": "paused",
      "progress": 0,
      "playbackRate": 1,
      "subtitle": null,
      "lastUpdateTime": 1736762400000,
      "currentProgress": 0
    },
    "participants": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "nickname": "房主小明",
        "role": "creator",
        "status": "online",
        "joinTime": "2026-01-13T10:00:00.000Z"
      }
    ],
    "creatorId": "550e8400-e29b-41d4-a716-446655440000",
    "createTime": "2026-01-13T10:00:00.000Z",
    "updateTime": "2026-01-13T10:00:00.000Z"
  },
  "creator": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "nickname": "房主小明",
    "role": "creator"
  }
},
"timestamp": "2026-01-13T10:00:00.000Z"
```

}

获取房间列表

获取所有可用房间的列表，支持分页和搜索筛选。

**请求方式：**GET

**请求路径：**/api/rooms

**查询参数：**

| 参数 | 类型 | 必填 | 说明 |
| --- | --- | --- | --- |
| keyword | string | 否 | 搜索关键词，匹配房间名称 / 房间号 |
| status | string | 否 | 房间状态过滤，可选值：waiting/playing |
| page | number | 否 | 页码，默认值 1 |
| pageSize | number | 否 | 每页数量，默认值 20 |

**成功响应：**


```
{
  "success": true,
  "code": 200,
  "message": "获取房间列表成功",
  "data": {
    "list": [
      {
        "id": "123456",
        "name": "周末观影派对",
        "capacity": 20,
        "currentCount": 5,
        "hasPassword": true,
        "status": "waiting",
        "creatorNickname": "房主小明",
        "createTime": "2026-01-13T10:00:00.000Z"
      }
    ],
```

"pagination": {
"page": 1,
"pageSize": 10,
"total": 2,
"totalPages": 1
}
},
"timestamp": "2026-01-13T10:30:00.000Z"
}

获取房间详情

获取指定房间的完整信息。

**请求方式：**GET

**请求路径：**/api/rooms/:roomId

**路径参数：**

| 参数 | 类型 | 必填 | 说明 |
| --- | --- | --- | --- |
| roomId | string | 是 | 6 位数字格式的房间 ID |

{
"success": true,
"code": 200,
"message": "获取房间详情成功",
"data": {
"id": "123456",
"name": "周末观影派对",
"capacity": 20,
"currentCount": 5,
"hasPassword": true,
"announcement": "欢迎来到我的观影室！",
"status": "waiting",
"videoState": {
"source": null,

"status": "paused",
"progress": 0,
"playbackRate": 1,
"subtitle": null,
"lastUpdateTime": 1736762400000,
"currentProgress": 0
},
"participants": [
{
"id": "550e8400-e29b-41d4-a716-446655440000",
"nickname": "房主小明",
"role": "creator",
"status": "online",
"joinTime": "2026-01-13T10:00:00.000Z"
}
],
"creatorId": "550e8400-e29b-41d4-a716-446655440000",
"createTime": "2026-01-13T10:00:00.000Z",
"updateTime": "2026-01-13T10:00:00.000Z"
},
"timestamp": "2026-01-13T10:30:00.000Z"
}

验证房间密码

验证房间密码有效性（加入有密码房间前的预检查）。

**请求方式：**POST

**请求路径：**/api/rooms/:roomId/verify-password

**路径参数：**

| 参数 | 类型 | 必填 | 说明 |
|---|---|---|---|
| roomId | string | 是 | 6 位数字格式的房间 ID |

**请求体参数：**

| 参数 | 类型 | 必填 | 说明 |
| --- | --- | --- | --- |
| password | string | 是 | 待验证的房间密码 |

**成功响应（密码正确）：**


```
{
  "success": true,
  "code": 200,
  "message": "密码验证成功",
  "data": { "valid": true },
  "timestamp": "2026-01-13T10:30:00.000Z"
}
```


加入房间

加入指定的线上观影室。

**请求方式：**POST

**请求路径：**/api/rooms/:roomId/join

**路径参数：**

| 参数 | 类型 | 必填 | 说明 |
| --- | --- | --- | --- |
| roomId | string | 是 | 6 位数字格式的房间 ID |

**请求体参数：**

| 参数 | 类型 | 必填 | 说明 |
| --- | --- | --- | --- |
| nickname | string | 是 | 加入房间的用户昵称 |
| password | string | 否 | 房间密码（仅房间有密码时需传） |

**成功响应：**


```
{
  "success": true,
  "code": 200,
  "message": "加入房间成功",
  "data": {
    "room": {
      "id": "123456",
      "name": "周末观影派对",
```

"capacity": 20,
"currentCount": 6,
"hasPassword": true,
"announcement": "欢迎来到我的观影室！",
"status": "waiting",
"videoState": {},
"creatorId": "550e8400-e29b-41d4-a716-446655440000",
"createTime": "2026-01-13T10:00:00.000Z",
"updateTime": "2026-01-13T10:30:00.000Z"
},
"participant": {
"id": "660e8400-e29b-41d4-a716-446655440001",
"nickname": "观影小王",
"role": "viewer",
"status": "online",
"joinTime": "2026-01-13T10:30:00.000Z"
}
},
"timestamp": "2026-01-13T10:30:00.000Z"
}

退出房间

退出当前所在的观影房间。

**请求方式：**POST

**请求路径：**/api/rooms/:roomId/leave

**路径参数：**

| 参数 | 类型 | 必填 | 说明 |
|---|---|---|---|
| roomId | string | 是 | 6 位数字格式的房间 ID |

**请求体参数：**

| 参数 | 类型 | 必填 | 说明 |
|---|---|---|---|
| participantId | string | 是 | 加入房间时返回的参与者 ID |

**成功响应：**

{
"success": true,
"code": 200,
"message": "退出房间成功",
"data": null,
"timestamp": "2026-01-13T11:00:00.000Z"
}

解散房间

解散指定的观影房间（仅房间创建者可操作）。

**请求方式：**DELETE

**请求路径：**/api/rooms/:roomId

**路径参数：**

| 参数 | 类型 | 必填 | 说明 |
|---|---|---|---|
| roomId | string | 是 | 6 位数字格式的房间 ID |

**请求体参数：**

| 参数 | 类型 | 必填 | 说明 |
|---|---|---|---|
| operatorId | string | 是 | 操作者 ID（必须为房间创建者 ID） |

**成功响应：**

{
"success": true,
"code": 200,
"message": "房间已解散",
"data": null,
"timestamp": "2026-01-13T12:00:00.000Z"
}

获取房间统计信息

获取系统内房间的全局统计数据。

**请求方式：**GET

**请求路径：**/api/rooms/stats

**成功响应：**


```
{
  "success": true,
  "code": 200,
  "message": "获取统计信息成功",
  "data": {
    "totalRooms": 10,
    "totalParticipants": 45,
    "waitingRooms": 6,
    "playingRooms": 4
  },
  "timestamp": "2026-01-13T10:30:00.000Z"
}
```


### 1.2. Socket Events：同步控制模块

事件：加入同步频道

Client → Server：sync:join

Payload：{ roomId, participantId }

Ack：{ ok, data: { channel, videoState, serverTime } }

事件：统一同步事件广播

Server → Client：sync:event

Payload：


```
{
  "type": "PAUSE",
  "roomId": "123456",
  "operatorId": "uuid",
  "serverTime": 1730000000000,
  "payload": {
    "progress": 120.5,
    "playbackRate": 1.0,
    "subtitle": null,
```

```
    "source": "http://..."
  }
}
```

控制事件（Client → Server）

| 事件名 | 说明 | Payload | 权限 |
|---|---|---|---|
| sync:play | 播放 | {roomId, operatorId} | creator |
| sync:pause | 暂停 | {roomId, operatorId} | creator |
| sync:seek | 跳转 | {roomId, operatorId, progress} | creator |
| sync:changeRate | 倍速 | {roomId, operatorId, rate} | creator |
| sync:changeSubtitle | 字幕 | {roomId, operatorId, subtitle} | creator |
| sync:changeSource | 片源 | {roomId, operatorId, sourceUrl} | creator |

备注：第一次迭代默认“房主=控制者”

### 1.3. Socket Events：聊天互动模块

详细见 docs/API 文档.md 的“Socket.IO 实时接口（聊天子系统）”。本节摘录关键字段。

**Namespace**：/chat（与同步模块分离，职责清晰）

**Room Channel**：room:{roomId}（与同步模块一致，绑定房间维度）

**前置条件**：加入聊天频道前，必须先通过 HTTP API POST /api/rooms/:roomId/join 获取 participantId，非房间成员无法操作

| 事件方向 | 事件名 | 说明 |
|---|---|---|
| C→S | room:join | 加入聊天频道（必须是房间成员） |
| C→S | room:leave | 离开聊天频道 |
| C→S | message:send | 发送群聊消息（无历史） |

| | | |
|---|---|---|
| S→C | message:new | 广播新消息 |

## 6.5. 数据管理策略

### 1. 当前实现

存储方式：使用 JavaScript 的 Map 对象在内存中存储数据

存储位置：backend-online-movie-house/src/repositories/RoomRepository.js

数据结构：this.rooms = new Map() - 房间 ID 到 Room 实例的映射

每个 Room 实例包含参与者 Map：this.participants = new Map()

### 2. 数据模型

Room（房间）：聚合根，管理房间配置、视频状态和参与者

Participant（参与者）：房间内的用户，包含 ID、昵称、角色、状态等

VideoState（视频状态）：视频播放状态，包含播放进度、播放状态等

### 3. 设计考虑

第一迭代：快速原型，使用内存存储简化开发

可扩展性：Repository 模式为未来迁移到持久化存储（如 Redis、MongoDB）提供兼容接口

数据一致性：通过聚合根模式确保业务规则

### 4. 后续迭代计划

添加数据持久化（Redis、MongoDB 等）。

## 6.6. 软件实现标准

### 1. 代码命名规范

#### 1.1. JavaScript/TypeScript 类与文件

类名：采用帕斯卡命名法（PascalCase），例如 RoomService、UserAuthController

后端源文件：采用帕斯卡命名法，例如 RoomService.js、RoomController.js

前端源文件：采用全小写字母，单词间用连字符分隔，例如 room-service.ts、user-auth-controller.js

接口/抽象类：以 I 为前缀，例如 IRepository、IValidator

类型定义文件：以 .d.ts 结尾，例如 room-types.d.ts

#### 1.2. 成员变量与局部变量

私有成员变量：采用 _ 前缀加驼峰命名法（camelCase），例如 _roomList、

_currentUserId

保护成员变量：采用 _ 前缀，仅在需要子类访问时使用

公共成员变量：采用驼峰命名法，尽量避免直接暴露

局部变量与参数：采用驼峰命名法，例如 targetIndex、videoUrl

静态成员变量：采用驼峰命名法，例如 instanceCount、defaultConfig

### 1.3. 成员函数（操作）

方法名：采用驼峰命名法，动词或动宾结构，例如 calculateRecommendation、uploadFile

获取器：使用 get 前缀或属性名，例如 getUserName() 或 userName()

设置器：使用 set 前缀，例如 setUserName()

异步方法：明确标识，例如 async fetchRoomData() 或 validateUserAsync()

### 1.4. Vue.js 组件与属性

组件文件名：帕斯卡命名法，必须与组件名一致，例如 RoomView.vue、UserProfile.vue

组件名：帕斯卡命名法，例如 RoomView、UserProfile

组件属性：驼峰命名法，例如 props: { roomId: String, userName: String }

自定义事件：采用 kebab-case，例如 @user-joined、@video-paused

### 1.5. 数据库对象（后续迭代）

表名：全小写，复数形式，下划线分隔，例如 users、video_sessions

字段名：全小写，下划线分隔，例如 created_at、video_duration

索引名：idx_表名_字段名，例如 idx_users_email

## 2. 可见性与封装标准

### 2.1. JavaScript/TypeScript 访问控制

Private：所有非静态成员变量必须设为私有。外部对数据的访问必须通过公共接口进行

Protected：仅在设计模板方法模式或确需子类访问父类内部状态时使用

Public：仅暴露对外的业务操作接口。

### 2.2. Props：用于将父组件数据传递给子组件

项目中使用 TypeScript 接口定义 props，提供更好的类型安全。

父组件使用示例 (RoomView.vue):

(传递视频源和其他配置给 ArtPlayer 组件)

<ArtPlayer

```
  :src="roomInfo.videoUrl"
  class="video-player"
  airplay
  aspect-ratio
  auto-size
  auto-orientation
  （多个布尔类型 props 使用 kebab-case）
...
/>
```

子组件定义示例 (ArtPlayer.vue):

(使用 TypeScript 接口定义 props，提供完整类型支持)

```
const props = defineProps<{
  src: string;                    // 必需属性，视频源 URL
  airplay?: boolean;              // 可选属性，AirPlay 支持
  aspectRatio?: boolean;          // 可选属性，宽高比保持
  autoSize?: boolean;             // 可选属性，自动尺寸调整
  autoOrientation?: boolean;      // 可选属性，自动方向调整
  autoPlayback?: boolean;         // 可选属性，自动播放
  fastForward?: boolean;          // 可选属性，快进支持
  flip?: boolean;                 // 可选属性，画面翻转
  fullscreenWeb?: boolean;        // 可选属性，网页全屏
  lock?: boolean;                 // 可选属性，锁定控制
  loop?: boolean;                 // 可选属性，循环播放
  isLive?: boolean;               // 可选属性，直播模式
  muted?: boolean;                // 可选属性，静音
  miniProgressBar?: boolean;      // 可选属性，迷你进度条
  pip?: boolean;                  // 可选属性，画中画
  screenshot?: boolean;           // 可选属性，截图
```

```
  subtitleOffset?: boolean;      // 可选属性，字幕偏移
}>();
```

### 2.3. 项目特点

TypeScript 优先: 所有 props 使用 TypeScript 接口定义

明确的类型: 每个属性都有明确的类型标注

可选参数: 大多数配置属性都是可选的，有合理的默认值

命名约定: 组件属性使用 camelCase 定义，模板中使用 kebab-case

### 2.4. 与通用规范的差异

| 通用规范 | 项目实际实践 | 说明 |
| --- | --- | --- |
| 选项式定义 | TypeScript 接口定义 | 项目使用更先进的组合式 API + TypeScript |
| 混合类型 | 明确类型标注 | 每个属性都有明确的 string、boolean 等类型 |
| 默认值在选项中 | 默认值在逻辑中处理 | 可选属性通过?标识，默认值在业务逻辑中设置 |

### 2.5. 模块导出规范

项目严格遵循模块导出规范，混合使用默认导出和命名导出。

默认导出（主要类或组件）

后端示例 (RoomService.js):

```
/**
 * 房间服务类
 * @class RoomService
 * @singleton
 */
class RoomService {
  // 类实现
}
module.exports = RoomService; // CommonJS 默认导出
```

前端示例 (roomApi.ts):

```
// API 服务对象
export const roomApi = {
```

```
  getList: async (params) => { /* 实现 */ },
  getDetail: async (roomId) => { /* 实现 */ },
  create: async (data) => { /* 实现 */ },
  // ... 其他方法
}
export default roomApi; // ES 模块默认导出
```

导入方式:

```
import roomApi from '@/api/roomApi' // 默认导入
import RoomService from './room-service' // 假设的导入
```

命名导出（工具函数、常量、类型）

后端示例 (utils/IdGenerator.js):

```
/**
 * ID 生成工具类
 * @class IdGenerator
 */
class IdGenerator {
  /**
   * 生成房间 ID
   * @static
   * @returns {string} 房间 ID
   */
  static generateRoomId() {
    // 实现
  }
  /**
   * 生成 UUID
   * @static
   * @returns {string} UUID
   */
```

```
  static generateUUID() {
    // 实现
  }
}
module.exports = IdGenerator; // CommonJS 导出
```

前端示例 (roomApi.ts 中的接口定义):

```
// 命名导出接口
export interface Room {
  id: string
  name: string
  creatorId: string
  createdAt: string
  participantCount: number
  status: string
}

export interface JoinRoomRequest {
  roomId: string
  nickname: string
  password?: string
}

export interface RoomCreatePayload {
  name: string
  capacity?: number
  password?: string
  announcement?: string
  creatorNickname: string
}
```

```
// 命名导出函数
export const reinitializeApiClient = (): void => {
  apiClient = null
}
```

导入方式:

```
// 导入类型
import type { Room, JoinRoomRequest } from '@/api/roomApi'

// 导入函数和常量
import { reinitializeApiClient } from '@/api/roomApi'

// 混合导入
import roomApi, { type Room, reinitializeApiClient } from '@/api/roomApi'
```

### 2.6. 项目导出规范总结

单一职责: 每个文件只导出一个主要功能

类型分离: 类型定义与实现分离，便于维护

明确导入: 导入时明确指定需要的内容

向后兼容: 保持导出接口的稳定性

# 后记

正文内容，方正仿宋，小四，首行缩进。正文内容，方正仿宋，小四，首行缩进。正文内容，方正仿宋，小四，首行缩进。

# 参考文献

[1] Vue.js 团队. Vue.js 官方文档[EB/OL].

[2] Socket.IO 团队. Socket.IO 官方文档[EB/OL].

[3] ArtPlayer 团队. ArtPlayer.js 官方文档[EB/OL].

[4] Express 团队. Express.js 官方文档[EB/OL].

[5] UML 2.2 面向对象分析与设计(第 4 版)

[6] UML 面向对象建模与设计 (（美）布莱哈，（美）朗博著, Michael Blaha, 巴拉赫 etc.) (Z-Library)

[7] 用例建模 (（美）Kurt Bittner，（美）Ian Spence 著；姜昊，许青松译 etc.) (Z-Library)